

Section 1 — Product Architecture: Adversarial Review & Hardening

Scope (per user redirect): Focus on **Section 1 (Product Architecture)** only. Adversarially review it in isolation from many angles, harden it, then organize the result and commit it to git.

Context

The original Section 1 sketched a single-purpose, "press record and walk away" multitrack recorder with four pillars (Trust, Speed, Awareness, Recoverability), three personas, a four-mode flow, and a short non-goals list. Treated in isolation and attacked from multiple directions, it holds up directionally but has **material gaps** that would bite a real broadcast/reality-TV deployment. This document records the attack, then a hardened Section 1, then how it lands in git.

Part A — Adversarial Review (by angle)

Each finding: **severity** (Critical / High / Medium / Low) + the fix folded into Part B.

A1 — Competitive positioning (High)

Original states "inspired by Boom Recorder" but never says *why someone picks this over the alternatives*: Boom Recorder, Reaper-as-recorder, Metacorder, Sound Devices Wave Agent (metadata), or a hardware recorder (Sound Devices/Cantar/Aaton/Nomad). Without an explicit wedge, the product is "another recorder." **Fix:** add a positioning statement — software-defined, high-density (Dante-class) iso recorder with first-class metadata/naming/post hand-off and operational awareness, on commodity Apple Silicon. Differentiator = reliability + workflow + scale at software cost, not yet-another-feature-set.

A2 — The headline promise overclaims (Critical)

"Press record, walk away 12 hours, get perfect files" is **unfalsifiable and unsafe** as stated. A single Mac mini faces power loss, kernel panics, thermal throttle, OS auto-updates, sleep/App Nap, Dante network blips, disk-full. The promise is only credible when paired with explicit *system-level* dependencies and *measurable guarantees*. **Fix:** (1) replace the vague promise with **explicit product guarantees/SLOs** (below); (2) add a **System Reliability Dependencies** subsection (appliance posture, sleep/App Nap inhibition, OS-update/Time-Machine policy, UPS expectation, NTP policy).

A3 — Missing reliability dependencies the app itself must own (Critical)

The app must actively defend its own runtime: prevent **App Nap** and **system/display sleep** while armed/recording, refuse cloud/network/Time-Machine destinations (already noted for §7 but it's a *product* principle), and warn on low power / thermal pressure. None of this was in Section 1. **Fix:** new subsection "System Reliability Dependencies" + elevate "Capture is sacred" to a first-class principle.

A4 — Take-based vs. continuous paradigm tension (High)

Section 1 implicitly leans **film/scripted** (take/roll). Reality TV, live events, and broadcast are predominantly **continuous iso recording** with marker-based logging and rollover, not discrete takes. The product targets *both*, but Section 1 never names the tension — which would misshape the whole UX and data model. **Fix:** state both operating paradigms explicitly (Take mode vs Continuous mode) as a product-level concept.

A5 — Pre-record buffer is absent (High)

Every serious production recorder (Boom Recorder, Sound Devices) offers a **pre-record / pre-roll buffer** (capture N seconds *before* the operator hits record). Omitting it from the product mode model is a competitive and operational miss. **Fix:** add a **Standby (pre-record buffering)** state to the mode model.

A6 — No degraded/error state in the mode model (High)

The "if a subsystem fails, recording continues and is logged" philosophy has **no state to represent it**. Setup→Armed→Recording→Review can't express "Recording (Degraded): mirror lost / LTC lost / disk warning." **Fix:** add **Recording (Degraded)** as an explicit operating state and define the failure hierarchy.

A7 — Redundancy / single-point-of-failure (High)

"Walk away 12 hours" on one box ignores that broadcast/reality runs redundant recorders (A/B) and UPS. Section 1 doesn't state whether multi-machine redundancy is in product scope (even future). **Fix:** add a redundancy stance — mirrored disks (near-term, already in spec) + a documented "run a second instance as backup recorder" posture, with full A/B chase deferred but acknowledged.

A8 — Personas are incomplete and conflated (Medium)

"Sound mixer / A2" conflates roles; reality/live use a **central audio control room operator** and **sound utility**, plus the **rental/facility tech** who builds presets, the **producer/showrunner** who only wants the dashboard, and the **Dante/IT network admin**. Also: buyer (production company) ≠ user. **Fix:** expand and de-conflate personas; separate buyer from users.

A9 — Missing pillar: interoperability / post hand-off (Medium)

BWF/iXML/markers/CSV/daily-folders are a *core value* (post integration) but aren't reflected in the pillars. The pillar set under-weights "fits the production→post pipeline." **Fix:** add a fifth pillar — **Interoperability / clean hand-off to post**.

A10 — Confidence monitoring output is unaddressed (Medium, open decision)

"Input metering only" + spot-check playback omits **live confidence monitoring** (headphone/PFL/solo) that production-sound operators expect. This is a genuine product-scope decision, not an oversight to silently keep out. **Fix:** surface as an explicit **open product decision** (in-scope live monitoring vs. metering-only), with a recommendation.

A11 — "No mixer" vs. per-track gain inconsistency (Low)

Non-goals say "no mixer," but the product has per-track gain. Ambiguous. **Fix:** clarify gain is **record trim**, not a monitor mix bus.

A12 — System clock / time integrity (Medium)

Wall clock drives naming, daily folders, and midnight logic; NTP jumps / DST / timezone changes can corrupt names and confuse midnight detection (which §8 separates from LTC corruption). **Fix:** add a product-level time-integrity stance (recommend stable/NTP-disciplined-but-no-step clock during record; treat wall-clock and media-clock as distinct).

A13 — Appliance vs. general-purpose app (Medium)

Reliability is far higher if the target Mac mini runs as a **dedicated appliance** (auto-launch, auto-recover, minimal other software, kiosk-ish) rather than a general desktop app among many. This is a product-architecture choice that was never made. **Fix:** state the **appliance-first deployment posture** as the recommended operating model.

A14 — Accessibility / colorblind-safe metering / localization (Low)

"Modern UX" for international crews implies colorblind-safe meters, large-text legibility at a distance, and localization. Not mentioned. **Fix:** add a short UX-principles note at product level.

A15 — Guarantees are not measurable (Critical, ties A2)

"Trust" and "perfect files" can't gate development. Product architecture should declare testable guarantees that become acceptance gates. **Fix:** **Product Guarantees (SLOs)** subsection with falsifiable statements.

Part B — Hardened Section 1 (Product Architecture)

1.0 Positioning

A **software-defined, high-density isolated-track recorder** for Apple Silicon: Dante-class channel counts, first-class metadata/naming/post hand-off, and broadcast-grade operational awareness — at commodity-hardware cost. It competes with Boom Recorder/Metacorder/Reaper-as-recorder and offloads from hardware recorders where channel density, metadata flexibility, and dashboarding matter more than a self-contained device. **Wedge = reliability + workflow + scale, not feature count.**

1.1 What it is

A single-purpose, always-trustworthy recorder whose **prime directive is "Capture is sacred"**: the capture path must never drop a sample; *every other subsystem (UI, mirror, LTC, network, dashboard) is best-effort and may degrade, but never stops or corrupts capture.*

1.2 Product Guarantees (SLOs) — testable, become acceptance gates

- **G1 No-loss:** zero dropped samples under the supported load envelope (channel count × SR × bit depth on the certified device matrix).

- **G2 Crash-valid:** every record press yields a recoverable, openable file even on power loss / kill-9 mid-write.
- **G3 TC-accurate:** per-file start timecode and BWF `TimeReference` are sample-accurate when LTC is present.
- **G4 Degrade-don't-die:** any single non-capture subsystem failure leaves capture running and is logged + surfaced.
- **G5 Long-run-stable:** bounded memory/CPU/thermals across multi-hour and multi-day soak (no leaks, no drift-induced xruns).

1.3 Product Pillars

1. **Trust** — capture never glitches; guarantees above are gated by tests.
2. **Continuity / uptime** — built for multi-hour/multi-day long-form.
3. **Speed of setup** — presets, track templates, token naming, auto take/roll.
4. **Operational awareness (not alarm fatigue)** — leveled INFO/WARNING/CRITICAL with hysteresis + filtering.
5. **Recoverability** — any interruption yields valid, recoverable files.
6. **Interoperability** — clean hand-off to post (BWF/iXML, markers, CSV, daily folders).

1.4 Operating Paradigms (both supported)

- **Take mode** (film/scripted): discrete takes, roll/take/scene, slate-style metadata.
- **Continuous mode** (reality/live/broadcast): long continuous iso record, marker-based logging, time/size/TC rollover.

The UX and data model must serve both; the difference is primarily in counters, naming defaults, and the logging emphasis.

1.5 Operating Modes (state machine)

Setup → Armed → Standby(pre-record buffering) → Recording → Recording(Degraded) ↔ Recording → Stopped → Review

- **Standby/Pre-record:** rolling N-second pre-buffer so record captures audio *before* the press.
- **Recording(Degraded):** capture continues; a non-capture subsystem (mirror/LTC/disk-warning/device-loss) is impaired and surfaced.
- **Review:** spot-check playback, **mutually exclusive with Recording** (isolated engine).
- **Overlays:** **Lock Mode** (block config/routing/device/metadata edits) and **Record Safe Mode** (guard Stop/Quit/Close/Change-project/Change-destination), enforced in the command layer across *all* control surfaces.

1.6 Failure Hierarchy (defines G4)

- **Tier 0 — Sacred (must never stop/corrupt):** input capture, FIFO drain, primary-disk write, crash-valid headers.
- **Tier 1 — Important (degrade, keep recording):** LTC, mirror disk, metering, health monitor, event log.
- **Tier 2 — Peripheral (may pause/fail freely):** remote dashboard, exports, UI cosmetics, control-surface links.

1.7 System Reliability Dependencies (the app owns these)

- Inhibit **App Nap** and **system/display sleep** while Armed/Recording.
- **Destination policy:** local/external physical volumes only; refuse cloud-synced (OneDrive/iCloud/Dropbox), network, and Time-Machine-target volumes.
- **Power:** detect on-battery / low-power / thermal pressure and warn; recommend UPS for the appliance.
- **Time integrity:** treat wall-clock and media-clock as distinct; recommend a disciplined-but-non-stepping system clock during record; handle DST/timzone without breaking naming or midnight logic.
- **OS hygiene (deployment guidance):** disable auto OS updates and Time Machine on the record machine during productions.

1.8 Deployment Posture

Appliance-first: recommend running the target Mac mini M4 as a dedicated recorder appliance (auto-launch, auto-recover after restart, minimal other software, kiosk-like). It also runs as an ordinary app, but appliance mode is the reliability-optimal default.

1.9 Redundancy Stance

- **Near-term (in spec):** mirrored recording to a secondary disk (best-effort; primary never blocked).
- **Operational:** documented "run a second instance/machine as a backup recorder" posture.
- **Deferred:** full A/B chase-sync between instances — acknowledged, not in early scope.

1.10 Personas (de-conflated)

- **Buyer:** production company / facility (in-house first).
- **Production sound mixer / recordist** (Take mode, film/scripted).
- **Audio control-room operator + sound utility** (Continuous mode, reality/live).
- **Broadcast/OB engineer** (high-count Dante, LTC, long runs, disk planning).

- **Rental/facility tech** (builds device/track/session presets).
- **Producer / showrunner** (read-only remote dashboard consumer).
- **Dante / IT network admin** (network + clocking).
- **Post / DIT** (BWF/iXML, daily folders, markers, take logs).

1.11 UX Principles (product-level)

Modern macOS Dark/Light; **colorblind-safe, high-legibility metering** readable at a distance; consistent command model across keyboard/Stream Deck/OSC/HTTP; localization-ready.

1.12 Scope & Non-Goals

- **In scope:** capture, per-track **record-trim gain** + delay + bypass, input metering, markers, isolated spot-check playback, post-record exports, metadata/naming/folders, LTC awareness, health/recovery, mirrored record (near-term), remote dashboard (future).
- **Non-goals:** not a DAW; no editing; no plugins/effects beyond delay+gain; **per-track "gain" is record trim, not a monitor mix bus**; not a routing matrix/mixer.
- **Open product decision (A10):** *live confidence monitoring* (headphone/PFL/solo output) — currently metering-only. **Recommendation:** treat basic confidence monitoring (selectable solo/headphone bus) as in-scope for production-sound credibility, scheduled after MVP; confirm before locking the UX.

Part C — Execution: organize & commit to git

Requires leaving plan mode (commits are blocked in plan mode). On approval:

1. **Move the working tree out of OneDrive** is *not* required for this doc commit, but note the existing repo lives under a OneDrive path; committing a markdown doc is low-risk. (Full relocation remains a Phase-0 task.)
2. Create a docs structure in the repo:
 - `docs/architecture/01-product-architecture.md` ← hardened Section 1 (Part B).
 - `docs/architecture/reviews/01-product-architecture-adversarial-review.md` ← Part A (the attack log), for traceability.
 - `docs/architecture/README.md` ← index noting Sections 2–21 are pending the same treatment.
1. **Logical commit organization** (small, reviewable commits on a feature branch, since `main` is the working branch):
 - Branch: `docs/product-architecture` .
 - Commit 1: `docs: scaffold architecture docs structure (README/index)` .
 - Commit 2: `docs: add hardened product architecture (section 1)` .
 - Commit 3: `docs: add adversarial review log for section 1` .
1. Do **not** push automatically; leave pushing to the user (auto-push.sh exists but pushing is the user's call).

Verification

- `git log --oneline -5` shows the three logical commits on `docs/product-architecture` .
- `git status` clean; the new `docs/architecture/**` files tracked.
- Render the markdown to confirm tables/state-machine read correctly.
- Confirm no edits were made outside `docs/` (the recorder code does not exist yet).

Section 2 — Software Architecture: Adversarial Review & Hardening

Scope: Focus on **Section 2 (Software Architecture)** only. Attack it in isolation from many angles, then harden it.

Context (what Section 2 originally said)

A layered design — Presentation → Application/Command → Engine façade (publishes snapshots) → four engines (Capture RT, Recording, Disk Writer pool, LTC) → mockable Platform abstraction — with four cross-cutting rules: RT/non-RT firewall, one-way snapshot publication, single command choke point, everything mockable. Directionally sound, but several load-bearing details are unspecified or quietly wrong under attack.

Part A — Adversarial Review (by angle)

A1 — Threading model is named but not specified (Critical)

The diagram lists threads but defines no **thread inventory**, **priority policy**, or **non-blocking contract**. Unspecified: who owns each thread, real-time vs high vs normal priority, the rule that the RT thread *never* waits on any other thread, priority-inversion avoidance, and start/stop **shutdown ordering** (a classic use-after-free source). **Fix:** add an explicit Thread Inventory + priority/ownership/lifecycle table and the "RT waits on no one" invariant.

A2 — Two distinct planes are conflated: control vs data (Critical)

"Engine publishes snapshots at ~30 Hz" lumps slow status with the **hot meter/waveform data path**. At 128 tracks the meter plane wants direct lock-free per-track atomics, not a fat status struct copy. **Fix:** split a **Control/Status plane** (commands + status snapshots) from a **Data plane** (audio FIFOs + per-track meter atomics). Different mechanisms, different rates.

A3 — Snapshot mechanism + memory reclamation underspecified (High)

"Double-buffered / atomic seqlock" hand-waves the hard part: seqlock readers spin; pointer-swap double-buffering needs safe reclamation (ABA / who frees the old snapshot). **Fix:** pin a concrete, RT-safe scheme — **triple-buffer with atomic publish index** (no reader spin, no allocation, no free-after-read race) for status; raw atomics for meters.

A4 — How a command reaches the RT thread is missing (High)

"Commands go through the dispatcher" stops at the firewall. Live changes (arm, set gain/delay, bypass) must cross into the RT thread **without locks**. **Fix:** define a **lock-free command FIFO** consumed by the engine at buffer boundaries, plus per-parameter atomics for cheap continuous values (gain/bypass). No control path takes a lock the RT thread could contend.

A5 — No event/error bus; error model undefined (High)

EventLog and HealthMonitor exist as modules, but Section 2 shows **no channel** by which a writer-thread disk-full or an LTC dropout *reaches* them, and no error-handling strategy (exceptions are forbidden in RT). Status snapshots are one-way and continuous; discrete **events** (errors, markers, LTC events, rollover) need their own path. **Fix:** add an explicit **lock-free event bus** (engine → app) for discrete events, and a stated error model: `Result` /error-code returns at boundaries, **no exceptions across the firewall or in RT**.

A6 — No TimeAuthority / Transport component (High)

Sample-clock ↔ wall-clock ↔ timecode mapping is the backbone of BWF `TimeReference`, TC-rollover, markers, and midnight logic — yet "Clock" appears only as the *system* clock in the platform layer. There's no owner of the **sample-count master + sample↔TC↔wall map**. **Fix:** introduce a **Transport / TimeAuthority** component (engine-level) owning the monotonic sample master and the time-domain mapping; LTC/Recording/Marker subsystems consume it.

A7 — Layering inversion: engine writes the RecoveryJournal (Medium)

RecoveryJournal sits in the Application layer, but the **writer (engine layer) must update it during recording** — an engine→application dependency that violates the layering. **Fix:** define journal/event sinks as **interfaces injected downward** (dependency inversion); the engine depends on `IJournalSink` / `IEventSink`, not on app-layer classes.

A8 — Process model: a Tier-2 crash kills Tier-0 (High)

Everything is one process, so a fault in dashboard/export/UI can take down **capture** — directly contradicting "Capture is sacred." **Fix:** state a process-isolation policy — run **exports out-of-process** (ffmpeg child) now, and put the **remote dashboard** (and ideally control-API ingress) behind an **XPC/separate process** so Tier-2 cannot crash Tier-0. Keep MVP simple but reserve the seam.

A9 — FIFO overflow / backpressure policy undefined (High)

"Large buffers absorb disk stalls" is not a policy. What happens when a FIFO actually fills (disk can't keep up)? **Fix:** define the **overflow contract**: RT never blocks; on full FIFO it marks an **xrun event** (counted, surfaced, logged) and follows a stated drop policy; sizing = "seconds of headroom" with the overflow path exercised in tests, not assumed impossible.

A10 — Over-abstraction risk for a solo dev (Medium)

Facade + dispatcher + a dozen services is gold-plate-able. **Fix:** mark the **MVP-essential abstractions** (RT firewall, platform mocks, lock-free FIFO, snapshot/event bus, command FIFO) vs. **start-concrete-extract-later** (full service layer, facade richness). Build simple first; the firewall is the only abstraction that's non-negotiable from day one.

A11 — Single source of truth / change-notification absent (Medium)

Session/Metadata/Naming/Counters all hold state; a metadata change must update the naming preview, etc. No defined ownership or observer model → drift/inconsistency. **Fix:** name a **SessionModel** as single source of truth on the message thread with a change-notification (observer) mechanism feeding dependent services and the UI.

A12 — Schema versioning / migration not architectural (Medium)

JSON sessions/presets and the SQLite journal need **version fields + migration** from v1, or early files break later. **Fix:** add schema-version + forward-migration as a platform/storage concern.

A13 — Control-API trust boundary missing (Medium)

Dashboard is read-only (good), but **OSC/HTTP control is read-write into Tier-0 commands**. No auth/bind-scope/rate-limit/authz stated. **Fix:** define a **network trust boundary**: control ingress is authenticated, LAN-bound, rate-limited, routed through the *same* command choke point (so Record-Safe/Lock apply), and separated from the read-only dashboard surface.

A14 — Startup / recovery / device-acquire sequencing undefined (Medium)

No boot orchestration: detect prior crash → recovery decision → acquire device → restore session → (never auto-arm before recovery is resolved). **Fix:** add a **lifecycle/orchestration** description with explicit ordering and "no auto-arm before recovery decision."

A15 — Engineering observability seam not called out (Medium)

RT-callback duration histograms, FIFO high-watermarks, writer latency feed both tests and HealthMonitor, but aren't an architectural cross-cutting concern. **Fix:** add a **telemetry seam** (lock-free counters/histograms) as the shared source for internal metrics, tests, and user-facing health.

A16 — Stack alignment not stated (Low)

The threading/FIFO mechanisms should map to the chosen stack (JUICE: `AbstractFifo`, `AudioIODeviceCallback`, message thread) and note deviations (custom writer threads vs `ThreadPool`). **Fix:** one note aligning the model to JUCE primitives and where we go custom.

Part B — Hardened Section 2 (Software Architecture)

2.1 Two planes

- **Data plane (hot):** audio input → per-track lock-free SPSC FIFOs → writer pool; per-track **meter atomics** read directly by UI/dashboard. No locks, no allocation, no syscalls.
- **Control/Status plane:** commands in (lock-free command FIFO + parameter atomics) and status out (triple-buffered status snapshot + lock-free **event bus** for discrete events).

2.2 Layers (with corrected dependency direction)

Presentation (GUI + read-only dashboard) → Application (SessionModel = single source of truth, command dispatcher with Record-Safe/Lock gating, services) → **Engine façade** → Engines (Capture RT, **Transport/TimeAuthority**, Recording, Disk Writer pool, LTC) → Platform abstraction (AudioDevice, SystemClock, FileSystem, DiskSpace, Storage[SQLite/JSON, versioned], Net). **Engines depend on injected interfaces (`IEventSink`, `IJournalSink`) — never on app-layer classes.**

2.3 Thread inventory & invariants

Thread	Priority	Owns	Blocking allowed?
Audio callback	Real-time (CoreAudio)	capture, delay/gain, meter+clip taps, FIFO push, consume command FIFO at buffer edge	Never waits on any other thread
Writer pool (2–4)	High (non-RT)	FIFO drain, format, write, header patch, journal sink	May do I/O; never blocks RT
LTC decode	High	LTC tap → decode → events	No
Monitor/health	Normal	telemetry, leveling, snapshots	No
Message/UI	Normal	SessionModel, dispatch, rendering	Yes (off RT)
Net/dashboard	Low	read-only serve; control ingress via dispatcher	Yes (isolated)

Invariants: RT waits on no one; no exceptions in RT or across the firewall; no allocation after Arm; defined thread **start/stop ordering** (acquire device last on start, release first on stop) to prevent use-after-free.

2.4 Cross-cutting concerns (added)

- **Event/error model:** discrete events via lock-free event bus; `Result` /error-codes at boundaries; errors classified by **failure tier** (Sacred/Important/Peripheral from §1.6).
- **Backpressure:** FIFO-full $\Rightarrow$ RT marks counted xrun event + stated drop policy; never blocks.
- **TimeAuthority:** monotonic sample master + sample $\leftrightarrow$ TC $\leftrightarrow$ wall map; sole source for `TimeReference`, rollover, markers, midnight.
- **Telemetry seam:** lock-free counters/histograms feeding tests + health.
- **Persistence:** versioned schemas + migration; `SessionModel` as single source of truth with change-notification.
- **Network trust boundary:** authenticated, LAN-bound, rate-limited control ingress through the one command choke point; read-only dashboard separated.
- **Process isolation:** exports out-of-process now; dashboard/control ingress behind XPC/separate process so Tier-2 can't crash Tier-0.
- **Lifecycle:** boot = detect crash $\rightarrow$ recovery decision $\rightarrow$ acquire device $\rightarrow$ restore session; **no auto-arm before recovery resolved**.

2.5 Solo-dev pragmatism

Non-negotiable from day one: **RT firewall, lock-free FIFO, command FIFO + status/event channels, platform mocks, telemetry seam**. Start concrete and extract later: façade richness, full service layer, XPC isolation (reserve the seam, implement when dashboard/exports land). Don't gold-plate the layering before the capture tracer-bullet proves out.

Verification (Section 2)

- Walkthrough: trace a "disk full at 128ch while recording with LTC" event end-to-end — RT marks xrun $\rightarrow$ event bus $\rightarrow$ health leveling $\rightarrow$ UI/dashboard + EventLog + journal — with **capture never stalling**. The hardened model must express every hop without a lock on the RT thread.

Section 3 — Module Breakdown: Adversarial Review & Hardening

Scope: Focus on **Section 3 (Module Breakdown)** only. Attack the module list in isolation, then re-cut it.

Context (what Section 3 originally said)

A flat 19-row table (`AudioDeviceManager`, `CaptureEngine`, `RecordingEngine`, `DiskWriterPool`, `LTCEngine`, `MeteringService`, `MetadataService`, `NamingEngine`, `CounterService`, `SessionStore`, `RecoveryJournal`, `HealthMonitor`, `EventLog`, `MarkerService`, `ControlSurfaces`, `RemoteDashboard`, `PlaybackEngine`, `ExportService`, `GUI/ViewModels`) with a "responsibility" and a loose "thread context" column. Useful inventory, but it predates the §2 hardening, mixes altitudes, and hides what's reliability-critical.

Part A — Adversarial Review (by angle)

A1 — Modules implied by hardened §1/§2 are missing (Critical)

The breakdown doesn't include components the design now requires: **Transport/TimeAuthority**, **EventBus**, **CommandService** (the one choke point with Record-Safe/Lock gating — currently buried inside "ControlSurfaces"), **SessionModel** (in-memory single source of truth — distinct from the persistence-only "SessionStore"), **TelemetryService**, **PreRecordBuffer** (standby/pre-roll), **Destination/VolumePolicy** (local-only, cloud/network refusal, disk planning), **MirrorWriter**, **PresetService/TemplateService** (lumped into SessionStore), and an **AppCoordinator** (composition root / lifecycle). Optional **MonitoringOutput** if confidence monitoring is approved (§1 open decision). **Fix:** add these as first-class modules.

A2 — Altitude/granularity is inconsistent; criticality invisible (High)

A tiny RT-critical `CaptureEngine` sits in the same flat list as the huge `GUI/ViewModels`, with no grouping by **failure tier** (§1.6) or **plane** (§2.1). You can't see at a glance what must never fail. **Fix:** regroup by **Tier (0/1/2)**, and tag **plane + RT-safety class** per module.

A3 — Overlapping responsibilities / fuzzy ownership (High)

- **Crash-valid headers:** `DiskWriterPool` (header patching) vs `RecoveryJournal` — who owns recoverability?

- **Rollover:** RecordingEngine "orchestration" vs DiskWriterPool "gapless rollover" — decide vs execute unclear.
- **Metering/clip:** the RT *tap* lives in CaptureEngine, aggregation in MeteringService, leveling in HealthMonitor — three owners, fuzzy seam.
- **Filename resolution:** NamingEngine + CounterService + MetadataService all feed it — which one *owns* "produce the final path"?

Fix: state explicit **decide-vs-execute** / **tap-vs-aggregate-vs-level** boundaries per pair.

A4 — No dependency/interaction map (High)

A table can't be validated against §2's "no engine→app dependency" and Tier rules without showing **who talks to whom**. **Fix:** add dependency rules + a short interaction list; assert **no Tier-0 module depends on a Tier-2 module**.

A5 — Thread-context column is vague and omits RT-safety class (Medium)

"Any → serialized", "Message + control" don't say whether a module may be touched from the RT thread. (e.g., EventLog must be **never called from RT** — RT only posts to EventBus.) **Fix:** replace with **thread owner + RT-safety class** (RT-safe / non-RT / never-from-RT).

A6 — Test strategy per module not indicated (Medium)

Pure/deterministic modules (NamingEngine, CounterService, TimeAuthority math) deserve exhaustive unit tests; hardware-bound ones (AudioDeviceManager) need mocks + soak. The table doesn't distinguish. **Fix:** tag each module's **primary test strategy**.

A7 — No lifecycle owner / composition root (Medium)

§2 added boot/recovery sequencing and DI, but no module owns it. **Fix:** add **AppCoordinator** (composition root: builds graph, injects interfaces, owns start/stop ordering, drives the boot→recovery→acquire→restore sequence).

A8 — Process-model decisions not reflected (Medium)

ExportService is shown in-process ("background worker"); §2 hardened it to **out-of-process ffmpeg**. RemoteDashboard likewise XPC-isolated. **Fix:** update both to reflect supervision of isolated child processes.

A9 — ControlSurfaces conflates trust levels (Medium)

Keyboard + Stream Deck + OSC + HTTP in one module spans local input and **network ingress** with very different trust. The §2 network trust boundary has no owner. **Fix:** split **LocalControl** (keyboard / local Stream Deck) from **NetworkControlGateway** (OSC/HTTP: auth, bind-scope, rate-limit); both funnel into **CommandService**.

A10 — Too many modules for a solo dev to scaffold up front (Medium, YAGNI)

~27 modules after additions; MVP (§18) needs a fraction. **Fix:** tag each module with **phase** (MVP / TC / Workflow / Hardening / Later) per §17 so v1 stays small.

A11 — PlaybackEngine device contention undefined (Low/Med)

"Separate audio path" — does it share AudioDeviceManager or open its own device, and who enforces "not while armed"? **Fix:** PlaybackEngine acquires its **own output device**, mutually exclusive with capture; AppCoordinator enforces.

A12 — Naming/suffix convention loose (Low)

SessionStore vs needed SessionModel; mixed Engine/Service/Manager suffixes. **Fix:** convention — **Engine** = RT/near-RT subsystem; **Service** = message-thread logic; **Manager** = resource/lifecycle; **Model** = state.

Part B — Hardened Section 3 (Module Breakdown, regrouped)

Columns: **Tier** (§1.6) · **Plane** (D=data, C=control/status) · **RT class** (RT=runs in callback, NR=non-RT, NFR=never-from-RT) · **Phase**.

Tier 0 — Sacred (capture must never stop/corrupt)

Module	Plane	RT	Phase	Responsibility (boundary)	Test
AudioDeviceManager	C	NR	MVP	device discovery/select/validate, hot-plug, SR/clock monitoring, channel map	mock + hardware
CaptureEngine	D	RT	MVP	callback: delay→gain→bypass→ meter/clip tap →FIFO push; consume command FIFO at buffer edge	RT-guard + soak
PreRecordBuffer	D	RT	TC	rolling pre-roll ring feeding record-start	unit + soak
Transport/TimeAuthority	C	RT-read	MVP	sample master + sample↔TC↔wall map (sole owner)	pure unit
DiskWriterPool	D	NR	MVP	FIFO drain→format(WAV/RF64/BWF)→large writes→ preallocate + crash-valid header patch ; <i>executes</i> rollover	mock-disk + golden
Destination/VolumePolicy	C	NR	MVP	local-only enforcement, cloud/network/TM refusal, disk planning/remaining-time	mock-fs unit

Tier 1 — Important (degrade, keep recording)

Module	Plane	RT	Phase	Responsibility (boundary)	Test
RecordingEngine	C	NR	MVP	arm/start/stop, segment lifecycle, decides rollover (writer executes), sample↔TC stamping	mock + soak
LTCEngine	C	NFR	TC	decode (liblbc), TC tracking, dropout/discontinuity/rate/midnight events	synthetic corpus
MeteringService	C	NFR	MVP	aggregate RT meter atomics → UI/dashboard rate (CaptureEngine taps, HealthMonitor levels)	unit
HealthMonitor	C	NFR	Hardening	sample CPU/RAM/disk/FIFO/device/LTC/clip; level w/ hysteresis	unit (fake metrics)
RecoveryJournal	C	NFR	Hardening	durable WAL of files/segments/markers; owns <i>journal</i> , writer owns <i>headers</i>	crash/kill tests
EventBus	C	RT→NR	MVP	lock-free engine→app discrete events	unit
EventLog	C	NFR	Hardening	append-only operational log (consumes EventBus)	unit
TelemetryService	C	RT→NR	MVP	lock-free counters/histograms (callback duration, FIFO HWM, writer latency)	unit
MirrorWriter	D	NR	Later	best-effort secondary write; failure never blocks primary	mock-disk

Tier 2 — Peripheral (may pause/fail freely; ideally isolated)

Module	Plane	RT	Phase	Responsibility	Test
SessionModel	C	NFR	MVP	in-memory single source of truth + change-notification	unit
SessionStore	C	NFR	Workflow	persistence (JSON, versioned + migration)	unit
PresetService / TemplateService	C	NFR	Workflow	session presets + track templates	unit
MetadataService	C	NFR	Workflow	field model, templates, live-edit rules, BWF/iXML mapping	unit
NamingEngine	C	NFR	Workflow	owns final-path resolution from tokens (pulls metadata+counters+TC); sanitize/dedupe/preview	pure unit/property
CounterService	C	NFR	Workflow	roll/take/segment + increment rules	pure unit
MarkerService	C	NFR	Hardening	marker capture (TC/ts/note), CSV/PDF export	unit
CommandService	C	NFR	MVP	the choke point: Record-Safe/Lock gating; all surfaces funnel here	unit
LocalControl	C	NFR	Hardening	keyboard / local Stream Deck → CommandService	unit
NetworkControlGateway	C	NFR	Later	OSC/HTTP ingress: auth, bind-scope, rate-limit → CommandService	unit + fuzz
RemoteDashboard	C	NFR	Later	read-only; XPC/isolated; serves snapshots	integration
PlaybackEngine	D	NR	Later	isolated spot-check; own output device , mutually exclusive w/ capture	integration
ExportService	C	NFR	Later	supervises out-of-process ffmpeg from originals	integration
GUI/ViewModels	C	NFR	MVP	bind to snapshots/events; send Commands	snapshot/UI
AppCoordinator	C	NFR	MVP	composition root: DI wiring, start/stop ordering, boot→recovery→acquire→restore	integration

Dependency rules

- **No Tier-0 module depends on a Tier-1 or Tier-2 module.** Tier-0 emits via EventBus/Telemetry interfaces only (`IEventSink` , `ITelemetrySink` , `IJournalSink`) — injected, never concrete app types.
- RT-class **RT/NFR** modules are **never called from the audio thread** except those marked RT; cross-thread contact is FIFO/atomic/EventBus only.
- All control surfaces (Local, Network, GUI) reach the engine **only** through CommandService.

Verification (Section 3)

- Cross-check every §2 cross-cutting concern maps to an owning module (TimeAuthority↔Transport, event bus↔EventBus, trust boundary↔NetworkControlGateway, lifecycle↔AppCoordinator, backpressure↔CaptureEngine+Telemetry).
 - Confirm the MVP-phase set is small (the Tier-0 row + SessionModel/CommandService/GUI/AppCoordinator) and nothing MVP-tagged depends on a Later-tagged module.
-

Section 4 — Risk Analysis: Adversarial Review & Hardening

Context

Original had a 12-row risk table (R1–R12) plus a throughput sanity note. Good start, but it conflates probability with impact, omits whole risk classes, and has no scoring or ownership.

Part A — Adversarial Review

- **A1 (High) No likelihood × impact scoring or residual risk.** A flat table can't prioritize. → Add Likelihood/Impact/Exposure + residual-after-mitigation columns.
- **A2 (High) Missing risk classes:** clock-domain drift between sample-clock and LTC; **CoreAudio buffer-size renegotiation** mid-session; macOS **App Nap/sleep/Gatekeeper/permission** prompts stealing focus mid-record; **thermal throttle on a fanless-feel mini under sustained 128ch**; **memory fragmentation over multi-day runs**; **JUCE/3rd-party version regressions**; **filesystem choice (APFS vs exFAT)** behavior on power loss; **NTP step / DST** corrupting naming; **solo-dev bus factor** (no second maintainer).
- **A3 (Med) No owner / no trigger / no detection per risk.** Mitigation listed, but not *how we detect it occurred* nor *who/what monitors it*. → Tie each risk to a HealthMonitor signal + EventLog code.
- **A4 (Med) No "accepted risks" register.** Some risks are deliberately accepted for v1 (e.g., no A/B chase). Should be explicit, not silent.
- **A5 (Med) Throughput note understates the real enemy.** Bandwidth is trivial; the note should foreground **latency-spike distribution (p99/p999 write latency)** and **FIFO high-watermark** as the true risk metric.
- **A6 (Low) No risk re-review cadence.** Risks should be revisited at each phase gate.

Part B — Hardened Section 4

Scoring: each risk = Likelihood(L/M/H) × Impact(L/M/H) → Exposure, with a detection signal, owner module, and residual rating.

ID	Risk	LxI	Detection signal	Owner	Residual
R1	RT-safety violation (alloc/lock in callback)	MxH	RT-guard trap; callback-duration p999	CaptureEngine/Telemetry	Low
R2	Disk latency spike / stall	HxH	FIFO high-watermark; write p99/p999	DiskWriterPool/Health	Med
R3	Device hot-unplug / SR change mid-record	MxH	CoreAudio property listener event	AudioDeviceManager	Med
R4	Aggregate-device clock drift	MxH	inter-device drift estimate	AudioDeviceManager	Med
R5	LTC decode error at edges	MxM	LTC discontinuity/rate events	LTCEngine	Low
R6	RF64/>4GB transition corruption	LxH	golden-file CI; size-threshold log	DiskWriterPool	Low
R7	Crash/power loss → unusable files	MxH	repair-on-launch outcome	RecoveryJournal	Low
R8	UI/dashboard back-pressure stalls engine	LxH	engine never reads UI; assert in tests	Engine façade	Low
R9	Thermal throttle at 128ch sustained	MxM	SMC temp/throttle telemetry	HealthMonitor	Med
R10	Solo-dev scope/maintenance/bus-factor	HxM	phase-gate burndown	AppCoordinator/owner	Med
R11	Naming collision / invalid path	MxM	dedupe/sanitize unit+property tests	NamingEngine	Low
R12	Mirror failure mishandled (future)	MxM	mirror-status heartbeat	MirrorWriter	Med
R13	NTP step / DST corrupts naming/midnight	MxM	wall-clock jump detector	TimeAuthority	Low
R14	macOS update/permission/sleep interrupts run	MxH	sleep/AppNap inhibitor + power state	AppCoordinator	Med
R15	Multi-day memory fragmentation/leak	MxH	RSS trend in soak	HealthMonitor/Telemetry	Med
R16	3rd-party (JUCE/libltpc) regression	LxM	pinned versions + CI golden	build	Low

Accepted risks (v1): no A/B chase-sync; single-process UI (Tier-2 isolation deferred per §2); aggregate-device support discouraged not blocked. **Cadence:** re-score at every phase gate; promote any "Med" residual touching Tier-0 to a blocking item.

Verification (S4)

Every R# maps to a detection signal that exists in HealthMonitor/Telemetry/EventLog; no Tier-0 risk left at residual ≥ Med without a phase-gate action.

Section 5 — Data Model: Adversarial Review & Hardening

Context

Original gave a Project-Session-Recording(Take)-FileSegment hierarchy plus Marker/EventLog/Counters, with JSON for config/presets and SQLite for journal/log/markers.

Part A — Adversarial Review

- **A1 (Critical) No schema versioning / migration** on any entity — early files break on upgrade (echoes §2 A12).
- **A2 (High) Continuous-mode mismatch.** The model is take-centric; reality/live **continuous** recording (per §1.4) has no clean representation — a 12-hour continuous record isn't really a "Take." → Need a **Recording** that supports both bounded takes and open-ended continuous spans, with segments as the real unit.
- **A3 (High) Identity & references fragile.** Implicit IDs; no stable UUIDs, no referential integrity rules, no "what happens to FileSegment if Track is deleted mid-session." → UUIDs everywhere + defined cascade/orphan rules.
- **A4 (High) Time representation ambiguous.** startTC + startSampleSinceMidnight + startWallClock can disagree; no single canonical time anchor or rate/df flags at the right level. → Canonical anchor = sample-count from TimeAuthority; TC/wall derived; store rate+df per Recording.
- **A5 (Med) No append-only/event-sourced separation.** Mixing durable recording facts (immutable once written) with mutable config in the same store risks corrupting history. → Separate **immutable recording ledger** (segments/markers/events, SQLite WAL) from **mutable session config** (JSON).
- **A6 (Med) Channel-map / routing under-modeled** for poly groups and source-name provenance (Dante name vs user name).
- **A7 (Med) No relationship to crash-recovery state machine** — the journal needs explicit per-segment status (open/closing/closed/finalized/orphaned).
- **A8 (Low) Custom metadata fields typing** unspecified (string vs typed); affects iXML mapping + validation.

Part B — Hardened Section 5

Two stores:

- **Mutable config (JSON, versioned):** Project, Session, DeviceConfig, TrackConfig, MetadataSet+templates, NamingTemplate, FolderTemplate, Preset, Counters.config.
- **Immutable ledger (SQLite WAL):** Recording, FileSegment, Marker, EventLogEntry, CounterAllocation.

Every entity: id: UUID, schemaVersion, createdAt. Migrations are forward-only with a version registry.

Canonical time: TimeAuthority sampleCount is the anchor; FileSegment.startSample is authoritative; TC and wall-clock are derived/recorded-for-reference. Recording { mode: take|continuous, frameRate, dropFrame, startSample, endSample?|open }.

Segment lifecycle status: open → closing → closed → finalized | orphaned (drives recovery).

Referential rules: TrackConfig edits during record apply to *future* segments only; deleting a track mid-session is blocked while armed; FileSegments reference Track by UUID + a frozen snapshot of name/source at open time (provenance preserved even if track later renamed).

Source provenance: TrackConfig stores { userName, sourceName (device/Dante), deviceChannel } distinctly.

Custom fields: typed (string|number|enum|date) for clean iXML mapping + validation.

Verification (S5)

Round-trip a session through save→quit→reload with a schema bump and confirm migration; simulate kill-9 mid-segment and confirm ledger status + recovery reconstruct a valid file.

Section 6 — Audio Pipeline: Adversarial Review & Hardening

Context

Original: single CoreAudio device (aggregate discouraged), allocate-everything-at-arm, RT callback does delay→gain→bypass→meter/clip tap→FIFO push, with sample-accurate LTC↔sample stamping; meter post-gain.

Part A — Adversarial Review

- **A1 (Critical) Variable/!=expected buffer size & SR changes** not handled in the callback contract. CoreAudio can deliver variable frame counts and renegotiate. Pre-sized buffers assuming fixed N break. → Callback must handle any frame count ≤ a pre-allocated max; SR change triggers a controlled stop+re-arm, never an in-place realloc.
- **A2 (High) Pre-record buffer absent from the pipeline** (added in §1.5/§3) — must be a continuously-running ring even when not recording. → Standby fills PreRecordBuffer; on record, drain pre-roll into segment with sample-accurate stitch.
- **A3 (High) Delay 0–500ms memory at 128ch.** 500ms @ 192kHz @ 128ch @ float = ~49 MB — fine, but must be allocated at arm and **zeroed** to avoid garbage on first reads; delay-change-while-recording policy undefined (click-free?). → Delay changes apply at segment boundaries or with a crossfade; never realloc in RT.
- **A4 (High) Gain smoothing & dB→linear** must be precomputed and ramped to avoid zipper noise; bypass must be click-free. → One-pole/linear ramp per block; bypass = ramped, not hard switch.
- **A5 (Med) Meter tap point is a policy decision, not a default.** Post-gain reflects recording; many ops want **pre-fader input** to see true input. → Provide both taps; default = recorded (post-gain) with a switch (ties §1 A10).
- **A6 (Med) Multi-device reality.** "Single device" is clean but Dante (64ch) + built-in LTC input is a common 2-device need. Aggregate has drift (R4). → Support a **primary capture device + a separate, independently-clocked LTC/aux input** read on its own callback, correlated via TimeAuthority — avoids aggregating the main capture clock.
- **A7 (Med) Denormals** can spike CPU in delay/gain math. → Enable FTZ/DAZ on the audio thread.
- **A8 (Low) Channel count > device channels / sparse arming** mapping validation belongs here.

Part B — Hardened Section 6

Device model: one **primary clocked capture device**; optional **secondary aux device** (LTC/built-in) on its own callback, never aggregated into the capture clock; both correlated through **TimeAuthority** sample stamping.

Arm-time allocation (RT-free thereafter): per-track delay lines (max 500ms @ session SR, zeroed), gain ramp state, capture SPSC FIFOs (seconds of headroom), PreRecordBuffer ring, meter/clip atomics.

Callback (handles any frame count ≤ max; FTZ/DAZ on):

```

for each armed track:
  read source channel(s)
  DELAY (read offset; changes only at boundary/crossfade)
  GAIN (ramped dB→lin) [BYPASS = ramped]
  meter+clip tap (post-gain default; pre-fader tap optional)
  push → capture FIFO (and always → PreRecordBuffer in Standby/Record)
  stamp sampleCount → TimeAuthority; publish FIFO HWM + callback duration → Telemetry

```

SR/buffer change: detected on message thread → controlled stop + re-arm (no in-place realloc); logged as event. **No alloc/lock/syscall/exception in callback** (enforced by RT-guard).

Verification (S6)

Feed variable frame sizes + a mid-session SR change via the mock device; assert no realloc, click-free gain/bypass ramps (spectral check), correct pre-roll stitch, and FTZ active (denormal stress test shows no CPU spike).

Section 7 — Disk Writing Pipeline: Adversarial Review & Hardening

Context

Original: small writer pool (2–4), large aligned writes, F_PREALLOCATE, crash-valid header patching, bounded fsync, auto-RF64, gapless rollover (open-before-close), mirror best-effort.

Part A — Adversarial Review

- **A1 (Critical) Pool→file assignment & ordering** unspecified. 128 files across 2–4 threads needs a defined, contention-free partition and per-file write ordering guarantee (no interleaved partial frames). → Static partition: each file owned by exactly one writer; per-file monotonic write cursor.
- **A2 (High) fsync cadence vs latency spike conflict.** Periodic fsync *is itself* the latency spike (R2). → Decouple: data writes are frequent+buffered; **fsync on a separate cadence/thread**, and header-size patches use small targeted writes; consider `F_FULLFSYNC` only at finalize, not periodically.
- **A3 (High) Crash-valid header strategy under-specified for RF64.** Patching RIFF size + `ds64` mid-stream must be atomic-enough that a crash between data-write and size-patch still yields a recoverable file. → Write data first, advance a *known-good* size marker on a cadence; recovery computes true size from file length if patch lagged.
- **A4 (High) Auto-RF64 decision timing.** Deciding at open from a *projection* can be wrong (rollover changes duration). Switching WAV→RF64 mid-file is impossible. → Default to RF64-capable header when projected OR when rollover policy could exceed 4GB OR sample-rate/channel/duration unknown-but-large; otherwise plain WAV. Bias toward RF64 when uncertain.
- **A5 (Med) Gapless rollover sample-accuracy** needs the exact boundary sample defined by TimeAuthority (esp. for TC-boundary rollover) and metadata/counter carry — but also must not double-count or drop the boundary sample.
- **A6 (Med) Mirror write coupling.** If mirror shares the same FIFO consumption, a slow mirror disk back-pressures primary. → Mirror reads from its **own** tap/FIFO; mirror failure isolates fully.
- **A7 (Med) Disk-full handling** mid-write: must stop the affected destination gracefully, keep the other (mirror) going, finalize a valid truncated file, raise CRITICAL.
- **A8 (Low) Filesystem specifics:** APFS sparse/preallocation behavior, exFAT 4GB limit (forces RF64 / split), volume format detection feeds Destination policy.

Part B — Hardened Section 7

Pool: static file→writer partition (one owner per file, balanced by track count); per-file monotonic cursor; large aligned buffered writes.

Durability ladder: (1) frequent buffered data writes; (2) periodic **known-good size marker** + light header patch; (3) `fsync` on a separate low-priority cadence; (4) `F_FULLFSYNC` only at finalize. Recovery trusts file length when a patch lagged. **Format:** prefer **RF64 header when uncertain/large**; plain WAV only when provably <4GB across the rollover policy; exFAT volumes force RF64/split. **Rollover (gapless):**

TimeAuthority gives the exact boundary sample; open+preallocate next segment before closing current; carry metadata + increment segment counter; boundary sample written exactly once. **Mirror:** independent tap/FIFO/threads; failure raises status, never back-pressures primary.

Disk-full: finalize valid truncated file on the failed volume, continue others, raise CRITICAL + EventLog.

Verification (S7)

Kill-9 between data-write and size-patch → recovery yields a playable file; fill the primary volume mid-record → mirror continues, primary finalizes valid; force a TC-boundary rollover → byte-exact, no dropped/duplicated boundary sample (golden-file compare).

Section 8 — LTC Architecture: Adversarial Review & Hardening

Context

Original: liblrc decode off the RT thread via a dedicated tap FIFO; continuous decode; all 7 rates incl. DF/NDF; dropout/discontinuity/rate/midnight detection; sample-accurate `TimeReference`; jam-on-lock; event logging.

Part A — Adversarial Review

- **A1 (Critical) Sample-accurate audio↔TC correlation is the crux and is hand-waved.** "Stamp the sample index" must define *which* sample maps to *which* LTC frame edge, accounting for the LTC reader's internal latency and the aux-device clock offset (§6 A6). Error here = wrong `TimeReference`. → Define correlation at the **LTC frame-start edge**, store (sampleCount, tc) pairs, and fit a line to reject jitter.
- **A2 (High) Drift between LTC source clock and capture clock** (separate devices) accumulates over hours. A single offset isn't enough. → Continuously estimate **rate ratio** (LTC frames vs capture samples), not just offset; flag drift beyond tolerance.
- **A3 (High) Half-frame / direction / varispeed / jam vs free-run** edge cases. LTC can run backwards (rewind), pause, or varispeed; the decoder must not interpret these as corruption. → State a TC-state model: `locked, free-running(flywheel), paused, reverse, no-signal`; flywheel TC across short dropouts.
- **A4 (High) Midnight vs corruption disambiguation needs precise rule.** `23:59:59:max-00:00:00:00` valid only if it matches expected +1 frame at the detected rate; anything else = discontinuity. → Encode exact expected-next-frame arithmetic per rate/df.
- **A5 (Med) DF/NDF detection & mismatch with project setting.** Must auto-detect drop-frame from the stream (DF flag bit + skipped frame numbers) and reconcile with user setting; mismatch is a WARNING, not silent.
- **A6 (Med) LTC level/quality** (too hot/clipped, too low, wrong channel) — operator needs a confidence/quality indicator, not just lock/no-lock.
- **A7 (Med) What TC anchors a file** when LTC is absent at record start but arrives later, or vice-versa. → Define fallback: time-of-day from system clock with a "TC=ToD (no LTC)" flag; back-fill not allowed (immutable header) but logged.
- **A8 (Low) Frame-rate of 23.976/29.97** are non-integer; sample-per-frame is fractional → must use rational arithmetic, not rounded.

Part B — Hardened Section 8

Correlation: decode on a dedicated thread from an aux/selected-channel tap; for each frame, record `(sampleCount_at_frame_start, tc)` corrected for reader latency; **least-squares fit** over a sliding window yields offset **and rate-ratio** (drift), with jitter rejection. `TimeReference` (samples since midnight) computed from the fit at each segment's start sample using **rational** sample/frame math. **TC state machine:** `no-signal` → `locking` → `locked` → `flywheel(short loss)` → `discontinuity/rate-change/paused/reverse`, each emitting `EventBus` events; flywheel maintains TC across brief dropouts using the last good rate. **Midnight rule:** rollover accepted only when it equals the exact expected next frame for the detected rate+df; else discontinuity. **DF/NDF:** auto-detect from stream; mismatch vs project = WARNING. **Quality:** report LTC level/SNR/lock-confidence to `HealthMonitor`. **No-LTC fallback:** anchor to system ToD with explicit flag; immutable, logged.

Verification (S8)

Synthetic corpus: all 7 rates, drift (±50ppm), short dropouts (flywheel), reverse, pause, varispeed, exact midnight, near-midnight corruption, DF/NDF mismatch, hot/low level. Assert `TimeReference` sample-accuracy vs ground truth and correct state transitions/events.

Section 9 — Metadata Architecture: Adversarial Review & Hardening

Context

Original: typed field model; cascade Project→Session→Track; live-edit policy (future segments); BWF `bext` now, iXML designed-now-built-later; track names feed naming.

Part A — Adversarial Review

- **A1 (High) iXML "future" is a mistake for the broadcast audience.** iXML is the de-facto post hand-off standard (scene/take/track names, sync, notes); deferring it undercuts Pillar 6 (interoperability). → Promote iXML write to **early** (alongside BWF), at least the core schema (PROJECT/SCENE/TAKE/TAPE/NOTE/TRACK_LIST/SPEED).
- **A2 (High) bext is size-limited & encoding-constrained** (256-char Description, ASCII/Latin-1, fixed fields). Rich/Unicode metadata won't fit → must map cleanly: short canonical fields → bext, full set → iXML. → Define the explicit field→bext vs field→iXML mapping + truncation rules.
- **A3 (High) Live-edit concurrency.** Editing metadata on the message thread while writers embed headers per-segment needs a defined snapshot point: each segment freezes a metadata snapshot at open. → "Metadata applies at next segment open" with a frozen snapshot; current segment immutable.
- **A4 (Med) Poly vs mono metadata.** TRACK_LIST/CHANNEL mapping differs (one file many channels vs many files). iXML TRACK_LIST must reflect interleave order; per-mono-file gets its single channel entry.
- **A5 (Med) UMID / OriginatorReference / CodingHistory** generation rules unspecified (uniqueness, recorder ID, format string). → Define generators.
- **A6 (Med) Validation & required-field policy** before record (e.g., empty Scene/Take) — warn vs block, configurable.
- **A7 (Low) Sidecar export** (CSV/JSON per daily folder) for NLEs/sound report apps not mentioned — cheap, high post value.
- **A8 (Low) Field i18n/encoding** for international crews → UTF-8 in iXML, transliterate for bext.

Part B — Hardened Section 9

Model: typed fields (`string|number|enum|date`), templates, cascade Project→Session→Track→(segment snapshot). **Embedding (early, not deferred):**

- **bext:** Description (≤256, transliterated), Originator, OriginatorReference (generated), OriginationDate/Time, **TimeReference** (from TimeAuthority), CodingHistory, UMID.
- **iXML (core, early):** PROJECT, SCENE, TAKE, TAPE/ROLL, NOTE, SPEED (rate/df/TimeReference), TRACK_LIST (interleave-correct for poly; single for mono), plus custom fields.
- **Sidecar:** per-daily-folder CSV/JSON sound report.

Live-edit: changes apply at **next segment open** via a frozen snapshot; current/written segments immutable. **Generators:** UMID/OriginatorReference/CodingHistory deterministic from recorder ID + session + timestamp. **Validation:** configurable required-field policy (warn default, block optional) checked at arm/record.

Verification (S9)

Validate output with **BWF MetaEdit + fprobe + an iXML-aware tool**; confirm bext/iXML agreement, correct TRACK_LIST for mono vs poly, Unicode survives in iXML and is sanely transliterated in bext, and a mid-record metadata edit only affects subsequent segments.

Section 10 — Naming Engine: Adversarial Review & Hardening

Context

Original: one token engine for files + folders; ordered Token|Literal templates; tokens from metadata+counters+TC+device; visual drag-drop builder; live preview; sanitize; duplicate protection; folder templates with auto/ask/disable + midnight handling.

Part A — Adversarial Review

- **A1 (High) Token resolution timing & immutability.** A filename built from live counters/TC must be resolved **once** at segment open and frozen; re-resolving later (e.g., after a counter change) would rename or mismatch the journal. → Resolve-once-at-open, store the resolved name in the ledger.
- **A2 (High) Duplicate protection across rollover/mirror/crash-recovery.** Auto-suffix must be collision-safe against existing files on disk *and* the in-flight ledger *and* the mirror volume — not just within the session. → Check disk + ledger + mirror; reserve the name atomically before open.
- **A3 (High) Cross-platform / FS sanitization depth.** macOS allows `/` issues, leading dots, length limits (255 bytes UTF-8), reserved names, trailing spaces; exFAT/Windows interop (for delivery) bans `:*?"<>|`. → Sanitize to the **strictest common** profile when delivery target unknown; configurable.
- **A4 (Med) Empty/missing token values** (e.g., no Scene) produce ugly `___` or stray separators. → Define empty-token policy: collapse adjacent separators, optional placeholder, warn.
- **A5 (Med) Poly file naming vs per-track {TrackName}.** A poly file has many tracks; `{TrackName}` is ambiguous. → Token set scoped: file-level tokens vs track-level tokens; `{TrackName}` only valid in mono mode (validate template against mode).

- **A6 (Med) Counter/token interplay with TimeAuthority** for `{TcStart}` — must use the same frozen start sample as the file header (§7/§8).
- **A7 (Low) Template versioning/sharing** across presets; invalid-token detection when a template references a removed custom field.

Part B — Hardened Section 10

Resolution: at segment open, resolve template once against frozen {metadata snapshot, counter allocation, TimeAuthority TcStart, device}; store resolved path in ledger (immutable). `{TcStart}` shares the header's start sample. **Collision safety:** atomic name reservation checked against disk + ledger + mirror; deterministic suffix (`_dupN`) only after a real collision. **Sanitize:** strictest-common profile (macOS + Windows/exFAT delivery), 255-byte cap, reserved-name/leading-dot/trailing-space handling; configurable strictness. **Empty tokens:** collapse separators + optional placeholder + warn. **Mode-aware tokens:** file-level vs track-level token scopes; template validated against mono/poly and against available custom fields; live preview shows resolved + sanitized result with warnings. **Folders:** same engine; auto/ask/disable; midnight creates next daily folder without breaking an in-progress segment (new segment lands in new folder, prior segment stays).

Verification (S10)

Property tests: random metadata/counters → always FS-valid, never collides (incl. simulated existing files + mirror), poly templates reject `{TrackName}`, midnight crossing places segments correctly, `{TcStart}` matches the file's best TimeReference.

Section 11 — GUI Architecture: Adversarial Review & Hardening

Context

Original: MVVM bound to snapshots; main screen (status/TC/duration/LTC/device/disk/remaining + virtualized 128-row track list w/ name/delay/gain/meter/arm); meters from decimated atomics on ~30Hz timer; Dark/Light; Lock + Record-Safe enforced in command layer.

Part A — Adversarial Review

- **A1 (High) 128-row meter rendering performance** on the message thread can starve UI/cause dropped frames; JUCE component-per-cell at 128x(meter+controls) is heavy. → Virtualize *and* render meters in a single OpenGL/Metal-backed meter view (one component drawing all visible bars), not 128 components; cap redraw to visible rows.
- **A2 (High) Snapshot/meter consumption must never block or allocate on the UI tick** and must tolerate the engine racing ahead. → UI reads triple-buffered status + meter atomics; never locks; missed frames are fine (latest-wins).
- **A3 (High) Lock/Record-Safe UX correctness.** Graying buttons isn't enough (§2): but the GUI must also *reflect* gating state unambiguously and provide the deliberate unlock workflow without being defeated by keyboard focus. → Visible mode banners + confirm/hold-to-unlock; all actions route to CommandService which is the real gate.
- **A4 (Med) Accessibility/at-a-distance legibility** (§1.11): colorblind-safe meter palettes, large-mode, high-contrast, VoiceOver labels — not addressed.
- **A5 (Med) Error/event surfacing without alarm fatigue** (§12): the UI needs a non-modal, filterable notification area driven by the EventBus/HealthMonitor, never modal dialogs that block during record.
- **A6 (Med) Multi-window / external display** (control room often uses a big meter wall + an operator panel). → Support a detachable full-screen meter/status view.
- **A7 (Med) Live-edit fields while recording** must visibly indicate "applies to next segment" (§9 A3) to avoid operator confusion.
- **A8 (Low) State restoration / window layout persistence** and per-user layouts.

Part B — Hardened Section 11

Pattern: MVVM; ViewModels subscribe to triple-buffered status snapshots + the EventBus; commands out via CommandService only.

Rendering: virtualized track list (visible rows only); **GPU-backed single meter view** drawing all visible bars (Metal/OpenGL via JUCE) to hold 128 channels at a configurable 30–60Hz; UI tick is lock-free, alloc-free, latest-wins. **Main screen:** record status, TC (from TimeAuthority), duration, LTC status+quality, device status, disk remaining + remaining-time estimate, virtualized track list (name, source, delay, gain, meter, arm, clip). **Modes:** Lock + Record-Safe shown as persistent banners with deliberate **hold-to-unlock/confirm**; real enforcement in CommandService (covers keyboard/StreamDeck/OSC/HTTP). **Notifications:** non-modal, filterable, level-colored area fed by EventBus/HealthMonitor; **no blocking modals during record.** **Accessibility:** colorblind-safe palettes, high-contrast + large modes, VoiceOver labels, at-a-distance meter mode. **Windows:** detachable full-screen meter/status view for control-room displays; layout persistence. **Live-edit affordance:** fields editable during record show an "applies next segment" hint.

Verification (S11)

Drive 128 channels of synthetic meter data and confirm sustained 60Hz with no UI-thread allocation/locks (instrument the tick); attempt every protected action via UI/keyboard during Record-Safe and confirm all are gated by CommandService; colorblind palette + VoiceOver pass.

Section 12 — Health Monitoring: Adversarial Review & Hardening

Context

Original: sampler thread reads CPU/RAM/disk/FIFO/device/LTC/clip; INFO/WARNING/CRITICAL with hysteresis + dwell + rate-limit; clipping classified single/repeated/sustained; silence detection optional default-off; transitions logged.

Part A — Adversarial Review

- **A1 (High) "Health" must distinguish capture-fatal from cosmetic.** A CRITICAL on the dashboard ≠ a CRITICAL on FIFO overflow. Levels alone don't encode the **failure tier** (§1.6). → Cross every signal with its tier; only Tier-0/1 signals can raise CRITICAL that demands operator action.
- **A2 (High) The real leading indicator is FIFO high-watermark + write p99/p999**, not CPU%. CPU can look fine while a disk stall is about to overflow. → Make FIFO HWM and write-latency percentiles the primary Tier-0 health metrics with predictive (trend) warnings before overflow.
- **A3 (Med) Sampler cadence vs cost.** Sampling SMC temps / disk stats too often is itself overhead; too rarely misses spikes. → Tiered cadence: fast (FIFO/clip, ~10–20Hz via atomics, near-free) vs slow (CPU/RAM/temp/disk, ~1Hz).
- **A4 (Med) Hysteresis params per-signal, not global;** clip vs thermal vs disk need different enter/exit/dwell. → Per-signal threshold config + sane defaults.
- **A5 (Med) Alarm persistence & acknowledgement.** Operators need ack/snooze and a "still active" indication, plus a post-session health summary for the report. → Ack/snooze + latched-until-cleared + session health summary export.
- **A6 (Med) Clip classification needs sample-true detection** (inter-sample peaks / sustained-over-N-samples) from the RT tap, not RMS guesses. → True-peak-ish + consecutive-over-threshold counting in RT; classify off-thread.
- **A7 (Low) Self-health of the monitor** (did the sampler thread stall?) — watchdog.
- **A8 (Low) Thermal action policy:** warn only, or also suggest reducing channels/SR? Document recommended operator action per CRITICAL.

Part B — Hardened Section 12

Signal taxonomy: each signal tagged {tier, cadence-class, level-policy}. Tier-0 leading indicators = **FIFO HWM, write p99/p999, callback-duration p999, xrun count**; Tier-1 = device/LTC status, mirror status, RSS trend; Tier-2 = dashboard/export/network. **Cadence:** fast loop (atomics, ~10–20Hz) for FIFO/clip/callback; slow loop (~1Hz) for CPU/RAM/temp/disk. **Leveling:** per-signal enter/exit/dwell hysteresis + rate-limit/coalesce; **predictive** warnings on FIFO/latency trend before overflow. **Clipping:** RT counts consecutive-over-threshold + peak; off-thread classifies single/repeated/sustained → WARNING-system. Silence detection optional, default-off. **Operator UX:** ack/snooze, latched-active indication, recommended action per CRITICAL; **session health summary** appended to the sound report. **Self-watchdog:** monitor liveness check; missed sampler ticks raise a meta-warning.

Verification (S12)

Inject a slow-disk ramp and confirm a predictive WARNING fires *before* FIFO overflow; verify clip classifier on crafted waveforms; confirm a dashboard crash raises only a Tier-2 INFO/WARNING (never a capture-critical alarm); health summary exports.

Section 13 — Remote Dashboard: Adversarial Review & Hardening

Context

Original: read-only embedded HTTP + WebSocket/SSE on a low-priority thread serving the status snapshot; zero RT impact; LAN-bound; optional token auth; assets shareable with in-app web layer.

Part A — Adversarial Review

- **A1 (High) "Zero RT impact" must be structurally guaranteed, not promised.** An in-process server shares the address space with capture; a bug/leak/crash there can still hurt Tier-0 (contradicts §2 A8). → Run the dashboard **out-of-process (XPC/child)** reading a shared read-only snapshot (shared memory or a one-way pipe); a dashboard crash cannot touch capture.
- **A2 (High) Back-pressure & many clients.** Slow/many WebSocket clients must never queue unbounded in the capture process. → Server holds only the latest snapshot (drop-old), bounded per-client buffers, capped client count.
- **A3 (Med) Security surface.** "Optional token auth" on a broadcast LAN is weak; OB trucks/venues have hostile networks. → Default-on token, bind to explicit interface, optional TLS, read-only by construction (no control path), rate-limit, off-by-default.
- **A4 (Med) Snapshot rate vs cost.** Pushing full 128-ch meters to phones at 60Hz is wasteful/janky on cellular. → Adaptive: status at 1–4Hz, meters summarized (e.g., over/peak flags + reduced rate), client-negotiated.
- **A5 (Med) Discovery & resilience.** mDNS/Bonjour for finding the recorder; auto-reconnect; show "stale data" if the feed drops (never imply all-good when disconnected).
- **A6 (Low) Multi-recorder aggregation** (control room with several recorders) — design the snapshot schema to be recorder-identified for future aggregation.

Part B — Hardened Section 13

Process model: dashboard server is an **isolated child process**; it maps a **read-only latest-snapshot** (shared memory / one-way IPC) published by the engine; no IPC path allows control or write-back. A crash/leak there is invisible to capture. **Transport:** HTTP for assets, WebSocket/SSE for push; **latest-wins**, bounded per-client buffers, max-client cap; adaptive rates (status 1–4Hz, summarized meters); "stale/disconnected" state shown explicitly. **Security:** off by default; when on — token auth, explicit interface bind, optional TLS, rate-limit; structurally read-only. **Discovery:** Bonjour/mDNS advertise; client auto-reconnect. **Schema:** recorder-identified snapshot (future multi-recorder aggregation).

Verification (S13)

Kill the dashboard child mid-record → capture unaffected, auto-respawn; connect 50 throttled clients → capture-process memory/CPU flat; pull the network → clients show "stale," not green; confirm no endpoint can issue a command.

Section 14 — Playback / Spot-Check: Adversarial Review & Hardening

Context

Original recommendation: integrate as an isolated "Review" mode in the same app (not a separate binary), disabled while recording; own PlaybackEngine + audio path; select take, scrub, solo/group, cached-peak waveform; not a DAW; future companion via journal if needed.

Part A — Adversarial Review

- **A1 (High) "Same app" vs "Capture is sacred" tension.** Even disabled-during-record, playback code in-process can crash the recorder when *not* recording but a session is loaded. Risk is lower (not armed) but non-zero. → Acceptable for v1 *if* playback strictly cannot initialize while armed/recording and shares no RT structures; revisit out-of-process if instability appears.
- **A2 (High) Device contention** (§3 A11): playback output device vs capture input device. On one interface, opening output may renegotiate the device. → Playback acquires output **only after** capture is fully released; never co-resident; explicit mutual-exclusion in AppCoordinator.
- **A3 (Med) Reading files still being written** (spot-check a prior segment while recording continues) — the original disables all playback during record, but operators often want to verify the *last completed* take while still rolling. → Allow read-only playback of **finalized** segments on a separate output device during record, *if* hardware allows; otherwise disabled. Make it a capability check, not a blanket ban.
- **A4 (Med) Waveform peak generation cost** for long files (multi-hour) — must be cached/lazy/background, never block UI, never compete with writers. → Background peak cache, lazy per-view, throttled, lower priority than writers.
- **A5 (Low) Scope creep guard** — solo/group, scrub is fine; resist adding fades/edits/markers-editing that drift toward a DAW.

Part B — Hardened Section 14

Recommendation (confirmed, refined): integrated isolated Review mode, *not* a separate binary — single codebase for a solo dev, shares the session/ledger model directly. **Hard isolation:** separate PlaybackEngine, separate output-device acquisition, no shared RT structures, and AppCoordinator enforces mutual exclusion with capture. **During record:** allow read-only spot-check of **finalized** segments **only if** a distinct output path is available (capability-gated); never touch in-flight segments or the capture device; default off if hardware can't guarantee

isolation. **Waveform:** background, cached, lazy, throttled, lower-priority-than-writers peak generation. **Scope:** select/scrub/solo/group + optional waveform. Explicitly not an editor. **Escalation path:** if in-process playback ever proves destabilizing, promote to an out-of-process companion reading the ledger read-only (reserve the seam).

Verification (S14)

Confirm playback cannot initialize while armed; on a single interface, verify capture-release→playback-acquire ordering with no glitch; spot-check a finalized segment during an active record on a 2-device rig with zero capture impact; verify peak generation never raises writer latency.

Section 15 — Technology Stack: Adversarial Review & Hardening

Context

Original recommendation: **JUCE 8 + C++** single toolchain for engine + UI (custom LookAndFeel, Dark/Light), WebView reserved for dashboard; rejected SwiftUI-hybrid for a solo dev (still needs a C++ RT core + bridge); supporting: CMake, liblrc, SQLite, embedded HTTP, ffmpeg/AVFoundation, BWF MetaEdit/ffprobe/sox as test oracles.

Part A — Adversarial Review

- **A1 (High) The §11 GPU-meter + §13 out-of-process dashboard partly undercut the "JUICE for everything" simplicity.** GPU meters and XPC children are extra surfaces regardless of stack. → Still net-positive for JUICE (device handling, BWF/RF64, RT primitives), but acknowledge the UI isn't "free."
- **A2 (High) JUICE CoreAudio layer abstracts away some control** we need (clock source, Dante specifics, fine hot-plug semantics, aux-device correlation §6/§8). → Plan to drop to **raw CoreAudio HAL** for device management where JUICE is insufficient (a documented escape hatch), using JUICE for the rest.
- **A3 (Med) RF64 support in JUICE** is not guaranteed complete/robust → must validate early or write our own WAV/RF64/BWF writer (likely **write our own** for full control over crash-valid headers, §7). → Treat the file writer as **our code**, not JUICE's, for Tier-0 control.
- **A4 (Med) liblrc integration & licensing** (LGPL) — fine for in-house; verify for future distribution. SQLite (public domain) fine. JUICE license revisit before commercial (already noted §1/§19).
- **A5 (Med) Apple Silicon specifics:** real-time thread API (`os_workgroup` / Audio Workgroups for CoreAudio render threads), QoS classes, `THREAD_TIME_CONSTRAINT_POLICY` — must be used for writer/aux threads; ensure JUICE plays well or bypass. → Adopt **Audio Workgroups** for threads adjacent to the audio callback.
- **A6 (Med) Testing/CI on macOS** — GUI/audio CI is hard; need headless engine tests + a hardware soak rig. → Engine logic decoupled from JUICE UI for CI; hardware soak runs on the M4.
- **A7 (Low) Build reproducibility** — pin JUICE/liblrc/ffmpeg versions (R16).

Part B — Hardened Section 15

Confirmed: JUICE 8 + C++ single toolchain, with explicit escape hatches:

- **Device management:** JUICE by default; **raw CoreAudio HAL** where needed (clock source, hot-plug nuance, aux-device correlation, Audio Workgroups).
- **File writer:** **our own** WAV/RF64/BWF/iXML writer (Tier-0 control over crash-valid headers, preallocation, rollover), not JUICE's format writer.
- **RT threading:** adopt **Apple Audio Workgroups** + proper QoS/time-constraint for capture-adjacent threads.
- **UI:** JUICE components + custom LookAndFeel (Dark/Light) + **GPU-backed meter view**; WebView/out-of-process for dashboard.
- **Supporting:** CMake (pinned deps), liblrc (LGPL — verify for distribution), SQLite (PD), ffmpeg out-of-process for exports, BWF MetaEdit/ffprobe/sox/iXML tools as CI oracles.
- **Architecture-for-test:** engine logic decoupled from JUICE UI so it runs headless in CI; UI/audio integration validated on the M4 soak rig.

Documented alternative (still not recommended for solo v1): C++ RT core + SwiftUI shell via Obj-C++ bridge — best native UX, but doubles the surface; reconsider only if a team forms or native chrome becomes top-3.

Verification (S15)

Spike-test early: (1) raw-HAL device acquire + hot-plug on Dante/RME; (2) our RF64/BWF writer vs BWF MetaEdit/ffprobe golden; (3) Audio Workgroup adoption measurably tightens callback jitter; (4) headless engine test target builds in CI without the GUI.

Section 16 — Testing Strategy: Adversarial Review & Hardening

Context

Original: RT-safety harness (trap alloc/lock/syscall on audio thread); deterministic mock-based engine tests (fake device/clock/disk); LTC synthetic corpus; golden-file format validation (BWF MetaEdit/ffprobe/sox); hardware fault injection; multi-hour/day soak; property/fuzz tests; recovery (kill-9) tests.

Part A — Adversarial Review

- **A1 (High) No explicit acceptance gates tied to the §1.2 SLOs.** Tests exist but don't *gate* releases against G1–G5. → Define a gate matrix: each SLO ↔ the specific test that proves it ↔ pass criteria.
- **A2 (High) RT-guard can't catch everything** (priority inversion, lock-free correctness, ABA). → Add **ThreadSanitizer** runs, lock-free unit tests, and a dedicated **xrun/jitter measurement** harness (callback duration histogram, FIFO HWM under load).
- **A3 (High) Soak realism.** "Multi-hour" is vague; broadcast needs **multi-day continuous + thermal + power-loss** coverage with automated pass/fail (not eyeballing). → Define soak profiles (24h, 72h, 7-day) with automated invariant checks (zero xruns, RSS flat, all files valid).
- **A4 (Med) Determinism of the mock device** — must reproduce variable buffer sizes, SR changes, hot-plug, exact xrun injection, and be seedable for repeatability.
- **A5 (Med) Golden-file maintenance** — format goldens drift; need a regeneration+review process and external-tool validation in CI (containerized ffprobe/sox; BWF MetaEdit may need a Mac runner).
- **A6 (Med) Fault-injection coverage matrix** — enumerate the fault × state combinations (device pull during arm vs record vs rollover; disk full at start vs mid vs near-4GB) rather than ad hoc.
- **A7 (Med) Performance regression tracking** — callback jitter / write latency / CPU per channel must be tracked over time (perf CI), not one-off.
- **A8 (Low) UI testing** — snapshot/VM tests headless; manual checklist for the rest.

Part B — Hardened Section 16

Test pyramid + specialized rigs:

- **Unit/property (pure):** NamingEngine, CounterService, TimeAuthority math, metadata mapping, LTC frame arithmetic. Property + fuzz.
- **Lock-free correctness:** FIFO/command-queue/snapshot under ThreadSanitizer + stress loops.
- **RT-safety:** debug RT-guard (alloc/lock/syscall trap) + **jitter/xrun harness** (callback p999, FIFO HWM) under synthetic 128ch load.
- **Mock-integration:** seedable deterministic fake device (variable buffers, SR change, hot-plug, xrun injection) + fake failing/slow/full disk.
- **Format goldens:** our WAV/RF64/BWF/iXML vs BWF MetaEdit/ffprobe/sox in CI; documented regeneration+review.
- **LTC corpus:** all rates + drift/dropout/reverse/midnight/DF-NDF/level (§8).
- **Recovery:** kill-9 / simulated power-loss across the fault×state matrix → repair-on-launch validity.
- **Soak:** automated 24h / 72h / 7-day profiles on the M4 with invariant checks (zero xruns, RSS flat, thermal bounded, every file valid) + thermal + power-loss events.
- **Perf CI:** track callback jitter, write latency, CPU/channel over time; regressions block.

SLO gate matrix:

SLO	Proven by	Pass criterion
G1 no-loss	jitter/xrun harness + soak	0 dropped samples @ max load
G2 crash-valid	recovery fault×state matrix	100% files openable
G3 TC-accurate	LTC corpus golden	TimeReference sample-exact
G4 degrade-don't-die	fault injection	capture continues, event logged
G5 long-run-stable	7-day soak	RSS flat, 0 xruns, thermal OK

Verification (S16)

Each SLO gate is wired into CI/soak with an automated pass/fail; a deliberately-introduced regression (e.g., a malloc in the callback) is caught by the corresponding gate.

Section 17 — Development Roadmap: Adversarial Review & Hardening

Context

Original: Phase 0 foundation → 1 capture tracer-bullet → 2 MVP (UI+scale) → 3 TC → 4 workflow → 5 hardening → 6 control/review → 7 scale-out → 8 production readiness; each gated by RT-safety + soak.

Part A — Adversarial Review

- **A1 (High) MVP = "Capture + great UI" but hardening (recovery, hot-plug) is Phase 5** — that's *after* MVP. A "great UI" MVP that loses files on a device pull contradicts Priority-1. → Pull **core reliability** (device-loss survival, crash-valid files, basic recovery) *into* the MVP band, even if metadata/markers stay later.
- **A2 (High) Phases are feature-bands, not risk-ordered.** The riskiest unknowns (raw-HAL device handling, our RF64 writer, Audio Workgroups, 128ch jitter, LTC correlation) should be **spiked in Phase 0/1** to de-risk, not discovered in Phase 2–3. → Add explicit **de-risking spikes** up front.
- **A3 (Med) No calendar/effort sense for a solo dev** — phases give order but no rough sizing or parallelism limits. → Add coarse effort tiers + "one thing at a time" guidance.
- **A4 (Med) iXML moved earlier (§9) and out-of-process dashboard (§13)** shift phase content. → Reflect.
- **A5 (Med) No "vertical slice stays shippable" principle** — each phase should leave a usable, reliable build. → State it.
- **A6 (Low) Dependency between phases** (e.g., TimeAuthority needed before TC, metadata before naming) not drawn. → Note key prerequisites.

Part B — Hardened Section 17

Principle: risk-ordered, each phase ends shippable + passes RT-safety + a soak appropriate to its scope.

- **Phase 0 — Foundation + de-risking spikes (highest-risk-first):** move repo out of OneDrive; CMake+JUICE scaffold; CI w/ headless engine target; **RT-safety guard**; platform interfaces + seedable mock device. **Spikes:** raw-HAL device acquire/hot-plug (Dante/RME), our RF64/BWF writer vs goldens, Audio Workgroups jitter, 128ch FIFO/jitter under synthetic load. *Gate: spikes prove the risky unknowns.*
- **Phase 1 — Capture tracer-bullet:** device→delay/gain→FIFO→our writer→mono WAV (fixed count); arm/record/stop; Destination/VolumePolicy (local-only). *Gate: 1h zero-xrun soak, external-tool-valid file.*
- **Phase 2 — MVP (capture + great UI + core reliability):** GPU meter UI, virtualized track list, gain/delay/bypass, poly WAV, **RF64 auto + BWF + core iXML**, Dark/Light; **device-loss survival, crash-valid files, basic crash-recovery**; 64→128ch. *Gate: multi-hour high-count soak + device-pull/kill-9 recovery pass. ← MVP*
- **Phase 3 — Timecode (P2):** TimeAuthority hardening, LTC engine (all rates), correlation/drift, status+events, sample-accurate TimeReference, midnight.
- **Phase 4 — Workflow (P3/P5):** SessionModel/Store (versioned), metadata+templates, token naming+builder, daily folders, presets, take/roll/segment counters, rollover (duration/size/TC), pre-record buffer.
- **Phase 5 — Hardening+:** full recovery journal, health monitoring + clip detection, event log, mirror writer.
- **Phase 6 — Control & review:** markers (+CSV/PDF), keyboard + OSC/HTTP API + Stream Deck (via API), Record-Safe/Lock across all surfaces, isolated playback/review, out-of-process exports.
- **Phase 7 — Scale-out:** out-of-process remote dashboard, multi-recorder schema.
- **Phase 8 — Production readiness (§19).**

Prereqs: TimeAuthority → TC/rollover/naming; SessionModel → metadata → naming; VolumePolicy → recording. **Solo guidance:** one phase at a time; spikes may interleave; don't start Phase 4+ features until Phase 2 reliability gates hold.

Verification (§17)

Each phase has an explicit gate; Phase 0 spikes retire the top technical risks (R1–R6, A2) before feature build; every phase produces a shippable, soak-passing build.

Section 18 — MVP Recommendation: Adversarial Review & Hardening

Context

Original MVP = Phases 1–2: any CoreAudio device + channel map + missing-device detection; arm/record/stop; mono+poly WAV w/ RF64 auto + BWF; per-track delay/gain/bypass; modern virtualized metering UI; disk space + remaining-time; Dark/Light; local-volume guard; 64→128ch. Deferred: LTC, metadata/naming, recovery, health/clip, markers, control, dashboard, mirror, playback, exports.

Part A — Adversarial Review

- **A1 (Critical) Deferring recovery + device-loss survival from MVP violates Priority-1.** An MVP that drops files when Dante blips or on a crash isn't "reliable capture." → **Move device-loss survival + crash-valid files + basic recovery into MVP** (per §17 A1).
- **A2 (High) No TC at all in MVP** — but broadcast files without any timecode (even ToD) have limited use; full LTC can wait, but **ToD timestamp + BWF TimeReference from system clock** is cheap and should be in MVP. → Add ToD-based TimeReference (no LTC) to MVP.
- **A3 (Med) "Great UI" is subjective** — define MVP UI acceptance (128ch meters @60Hz, Dark/Light, no UI-thread stalls).
- **A4 (Med) MVP success metric must be the soak gate, not feature completeness** — restate DoD around G1/G2/G5 + the device-pull/kill-9 tests.
- **A5 (Low) Risk of MVP scope creep** from "great UI" — keep workflow (metadata/naming/markers) firmly out.

Part B — Hardened Section 18

MVP = Phase 1–2, reliability-first:

- Any CoreAudio device, channel map, missing-device detection, **hot-plug survival** (keep buffered audio, mark gap, re-acquire).
- Arm/record/stop; mono + poly WAV; **RF64 auto + BWF + core iXML; ToD-based TimeReference** (system clock, no LTC yet).
- Per-track delay/gain/bypass; GPU-backed 128ch metering @60Hz; Dark/Light; disk space + remaining-time; **local-volume-only guard**.
- **Crash-valid files + basic crash-recovery** (repair-on-launch).
- 64→128ch validated.

Deferred (next phases): full LTC, metadata/naming/templates/presets, take/roll, full recovery journal, health/clip system, markers, control surfaces, dashboard, mirror, playback, exports. **Definition of Done:** passes **G1 (0 xruns @128ch multi-hour)**, **G2 (kill-9 → valid files)**, **G5 (no RSS/thermal drift)** + a **device-pull-during-record** test (capture survives), with files validated by external tools.

Verification (S18)

Run the MVP DoD suite on the M4 with Dante: multi-hour 128ch soak (0 xruns), mid-record device pull (survives), kill-9 (recovers), output validated in BWF MetaEdit/ffprobe; UI sustains 60Hz with no thread stalls.

Section 19 — Production Readiness Roadmap: Adversarial Review & Hardening

Context

Original: reliability certification (72h+/multi-day soak); device matrix sign-off w/ fault injection; crash/power-loss recovery suite; packaging (Developer ID, hardened runtime, notarization, entitlements, updates, JUCE license/App-Store revisit); operational docs + crash reporting; beta with real productions.

Part A — Adversarial Review

- **A1 (High) "In-house first" still needs production-readiness — but a different bar.** The original reads like a commercial checklist; for in-house deployment the priority is **operational runbooks + appliance setup + recovery drills**, with commercial packaging deferred. → Split into **In-house GA** vs **Commercial GA** gates.
- ****A2 (High) No defined certification *evidence***** — "72h soak" needs recorded artifacts (logs, xrun counts, file-validation reports) that constitute a sign-off record, especially for broadcast trust. → Define a certification report artifact per release.
- **A3 (Med) Appliance deployment (§1.8) has no readiness items** — auto-launch, auto-recover-after-restart, sleep/update disabling, kiosk config, UPS guidance. → Add appliance hardening checklist.
- **A4 (Med) Field-failure feedback loop** — opt-in crash/telemetry + event-log export workflow so real-world failures improve the product. → Define.
- **A5 (Med) Data-safety / no-regression on originals** for exports (§ exports) and any future edit — guarantee originals never mutated. → Add as a release invariant.

- **A6 (Low) Localization/accessibility sign-off** for commercial; security review for network features before distribution.

Part B — Hardened Section 19

Two gates:

- **In-house GA (near-term):** pass all SLO gates (§16); device-matrix sign-off (DVS, Digiface, RME, Focusrite, USB/TB) incl. fault injection; **certification report artifact** (soak logs, xrun=0 evidence, recovery results, file-validation reports) per build; **appliance hardening checklist** (auto-launch, auto-recover-after-restart, sleep/AppNap/auto-update disabled, UPS, kiosk); operational **runbooks + recovery drills**; opt-in crash/telemetry + event-log export.
- **Commercial GA (later):** Developer ID signing, hardened runtime, **notarization**, entitlements (mic/audio), update mechanism; **JUCE commercial license** + App-Store/sandbox decision; **network-feature security review** (dashboard/OSC/HTTP); localization + accessibility sign-off; support/SLA story.

Release invariants: originals never mutated; every release ships a certification report; no Tier-0 risk at residual ≥ Med (§4).

Verification (S19)

Produce a signed-off certification report for an in-house GA candidate (soak + device matrix + recovery evidence); validate appliance survives a forced restart and resumes/recovers unattended.

Section 20 — Feature Prioritization: Adversarial Review & Hardening

Context

Original mapped features to the six engineering priorities (reliability→TC→metadata→scalability→workflow→UI) across phases, with explicit deferrals (mirror, dashboard, exports, OSC/MIDI breadth, iXML writing, App Store).

Part A — Adversarial Review

- **A1 (High) Internal inconsistencies after the deep review:** iXML moved to MVP/early (§9/§18), core reliability moved into MVP (§17/§18), confidence monitoring surfaced as in-scope-post-MVP (§1). The prioritization must be reconciled. → Update.
- **A2 (Med) Priorities are categories, not a backlog** — needs a should/could split (MoSCoW) within phases so trade-offs under time pressure are pre-decided. → Add Must/Should/Could/Won't per phase.
- **A3 (Med) No explicit "Won't (v1)"** beyond scattered deferrals — collect them so scope is unambiguous. → Consolidated Won't list.
- **A4 (Low) Confidence monitoring (§1 open decision)** needs a placed priority pending user confirmation.

Part B — Hardened Section 20 (reconciled, MoSCoW)

P1 Reliability (Must, Phases 0–2,5): RT firewall+harness, FIFO/writer-pool, our crash-valid writer, preallocation, **device-loss survival + crash-recovery in MVP**, soak gates, VolumePolicy. **P2 Timecode (Phase 3; Must: ToD TimeReference in MVP):** continuous LTC, correlation/drift, sample-accurate TimeReference, midnight, events. **P3 Metadata (Phase 2 core / Phase 4 full): BWF + core iXML early (Must),** full metadata model/templates/sidecar (Should), custom typed fields. **P4 Scalability (Phase 2, Must):** 128ch validated, virtualized GPU UI, writer-pool tuning. **P5 Workflow (Phases 4–6, Should):** naming/folders/presets/counters/rollover, pre-record buffer, markers, control surfaces (OSC/HTTP/Stream Deck), health monitoring, mirror. **P6 UI (Phase 2 Must shell; ongoing Should):** modern Dark/Light, meters; dashboard (Could, Phase 7). **Confidence monitoring (pending §1 decision):** if approved → **Should, post-MVP (Phase 5/6). Won't (v1):** A/B chase-sync; full DAW/editing; plugin/effects beyond delay+gain; in-process dashboard; MIDI control (eval only); App-Store distribution; aggregate-device as a supported high-count path.

Verification (S20)

Cross-check every feature's phase/MoSCoW against §17 phases and §1.6 tiers; confirm no Must depends on a Could; confirm the Won't list matches the deferrals scattered across §§1–19.

Section 21 — First Implementation Task: Adversarial Review & Hardening

Context

Original first task: Phase 0 foundation + Phase 1 capture tracer-bullet — (1) relocate out of OneDrive + scaffold CMake/JUCE/CI; (2) **RT-safety guard first**; (3) platform interfaces + mock device; (4) thinnest real path device→FIFO→writer→mono WAV on local volume w/ arm/record/stop; (5) prove via 1h zero-xrun soak + external-tool-valid file.

Part A — Adversarial Review

- **A1 (High) Missing the de-risking spikes (§17 A2)** that should precede or accompany the tracer-bullet: raw-HAL device handling, our RF64/BWF writer, Audio Workgroups, 128ch jitter. The first task should explicitly *retire top risks*, not just produce a happy-path recorder. → Fold the highest-risk spikes into the first task.
- **A2 (Med) "Thinnest real path" lacks the firewall proof** — the first task must demonstrate the RT/non-RT firewall + lock-free FIFO + Telemetry (FIFO HWM/jitter), not just write a file. → Make instrumentation part of the deliverable.
- **A3 (Med) Definition of done needs measurable criteria** beyond "1h soak" — specify channel count, xrun=0, FIFO HWM bounded, file validates, RT-guard clean under TSan.
- **A4 (Low) Sequence risk**: scaffolding before the firewall is fine, but the *first code that touches audio* must already be inside the firewall + guard.

Part B — Hardened Section 21 (the actual first task)

Goal: a reliability-proving vertical slice that retires the top technical risks.

1. **Foundation**: move repo out of OneDrive; CMake + JUCE scaffold; CI with a **headless engine test target**; pin deps.
2. **RT-safety guard first** (trap alloc/lock/syscall on the audio thread) — before any audio code.
3. **Platform interfaces + seedable mock device** (variable buffers, SR change, hot-plug, xrun injection).
4. **De-risking spikes (concurrent)**: (a) **raw-HAL** device acquire + hot-plug on Dante/RME; (b) **our WAV/RF64/BWF writer** validated vs BWF MetaEdit/ffprobe goldens; (c) **Audio Workgroups** on capture-adjacent threads; (d) **128ch jitter/FIFO** under synthetic load.
5. **Tracer-bullet path (inside the firewall)**: real device → lock-free FIFO → writer pool → mono WAV on a **local** volume; arm/record/stop; **Telemetry** (FIFO HWM, callback p999).
6. **Prove it (DoD)**: 128ch, 1h+ soak, xrun=0, FIFO HWM bounded, RT-guard + ThreadSanitizer clean, output file validates externally, hot-plug during record survives.

Verification (§21)

The first task is "done" only when the DoD above passes on the M4 with Dante — i.e., the riskiest unknowns (RT-safety, raw-HAL device handling, our format writer, jitter at 128ch, hot-plug survival) are demonstrably retired before any feature/UI work begins.

Planner Status

All 21 sections completed in deep format (adversarial review + hardened version + verification each). Sections 1–3 also include a separate hardened-rewrite; Sections 4–21 fold the hardening inline. Cross-section consistency reconciled in §17/§18/§20 (iXML-early, core-reliability-in-MVP). **RESOLVED DECISION (confidence/headphone monitoring, §1 A10): out of MVP; not in M1 or M2; assigned to Phase 5 / M5.** MVP is **metering-only**. Playback/spot-check is acceptable later (M5) but **no live monitor mixer, no PFL/solo bus, no headphone routing in MVP.**

PART II — EXECUTION PLAN (Architecture → Implementation)

Architecture approved in principle. This part plans the build. **No code.** It maps the approved architecture (Part I, §§1–21) onto features, milestones, modules, repo, tests, a first coding milestone, and a Claude Code session sequence.

Reconciliation of vocabulary: the user's "MVP = capable of [device/mono/poly/32tracks/meters/disk-time/LTC/basic-

metadata/naming/Record-Safe/Lock/markers/session-save-load]" is **broader** than the reliability-only MVP in §18. We honor the user's list: **MVP = Milestone 1 + Milestone 2**. Point-1's "Phase 2 / Phase 3" map to **Milestone 3 (Phase 2)** and **Milestones 4–5 (Phase 3)**.

1. Feature Triage

Legend — **MVP** = M1+M2 · **P2** = M3 · **P3** = M4–M5 · **Cut** = removed from roadmap.

Feature	Bucket	Notes / scoped depth
CoreAudio device select + channel map + missing-device detection	MVP	raw-HAL where JUCE insufficient (§15)
Device hot-plug survival (keep buffering, mark gap, re-acquire)	MVP	Priority-1; can't be deferred
Mono WAV / Polyphonic WAV	MVP	our writer (§7/§15)
RF64 auto + BWF bext	MVP	reliability + post hand-off
core iXML	MVP	promoted early (§9)
≥32 tracks (architected to 128)	MVP	MVP gate = 32; scale validated later
Input meters (GPU-backed)	MVP	§11
Disk remaining + remaining-time estimate	MVP	§ DiskMgmt
Local-volume-only guard (refuse cloud/network)	MVP	reliability
Crash-valid files + basic crash-recovery	MVP	Priority-1 (pulled in from §18)
LTC decode + display + sample-accurate TimeReference	MVP (M2)	drift/flywheel/quality → P2
Basic metadata (fields + cascade + bext/iXML map)	MVP (M2)	templates UI → P2
Naming templates (token engine + preview)	MVP (M2)	visual drag-drop builder polish → P2
Markers + notes (TC-stamped) + CSV export	MVP (M2)	PDF export → P2
Record Safe mode + Lock mode (CommandService-enforced)	MVP	§2/§11
Session save/load (SessionModel + versioned store)	MVP	§5
ToD-based TimeReference (no-LTC fallback)	MVP	§8 A7
Pre-record buffer	P2	valued, not MVP-critical
Auto rollover (duration/size/TC)	P2	M3
Session/track templates & presets	P2	M3
Take/Roll/Segment auto-numbering	P2	M3
Daily folder system (token-based)	P2	M3
Full health monitoring + clip classification	P2/P3	basic clip flag in MVP; full system M3/M4
Mirrored recording	P3 (M4)	best-effort, isolated
LTC drift/flywheel/reverse/quality	P2 (M3)	core decode is MVP
Visual naming builder (drag-drop)	P2 (M3)	functional templates in MVP
Stream Deck / OSC / HTTP control	P3 (M4)	keyboard shortcuts in MVP
Remote dashboard (out-of-process)	P3 (M5)	§13
Playback / spot-check (isolated Review)	P3 (M5)	§14
Exports MP3/AAC/FLAC (out-of-process)	P3 (M5)	never touch originals
Confidence/headphone monitoring	P3 (M5), pending decision	§1 A10 open
Multi-recorder dashboard aggregation	P3 (M5)	schema reserved
MIDI control	Cut	low value vs OSC/HTTP; re-eval only on demand
Aggregate-device as a <i>supported</i> high-count path	Cut	drift risk (R4); single clocked device only
A/B chase-sync between recorder instances	Cut (v-roadmap)	huge; out of scope; "run 2nd instance" stays operational-only
In-app waveform editing / fades / DAW features	Cut	non-goal (§1)

2. Realistic MVP Definition

MVP = a rock-solid 32-track recorder with thin-but-complete TC/metadata/workflow skin. Reliability and recording quality first; breadth deliberately shallow.

MVP does:

- Select any CoreAudio input device; map channels; detect missing device; **survive hot-plug** mid-record (buffer, mark gap, re-acquire).
- Arm/record/stop **≥32 tracks** as **mono and polyphonic WAV, RF64-auto + BWF + core iXML**, written by **our** crash-valid writer to **local volumes only**.
- **LTC decode** (the 7 rates) → live TC display + **sample-accurate BWF TimeReference**; **ToD fallback** when no LTC.
- **Basic metadata** (project/scene/take/roll/track/notes/custom) with cascade + bext/iXML mapping.
- **Naming templates** (token engine, live preview, sanitize, dedupe) — functional, not yet drag-drop.
- **Markers + notes** (TC-stamped) with CSV export; keyboard-triggerable.
- **Record Safe + Lock** modes enforced in CommandService.
- **Session save/load** (versioned) + **basic crash-recovery** (repair-on-launch).
- **GPU input meters, disk remaining + remaining-time**, Dark/Light.

MVP deliberately does NOT: pre-record buffer, rollover, templates/presets, daily folders, take/roll auto-numbering, mirror, dashboard, playback, exports, OSC/HTTP/Stream Deck, drag-drop naming UI, full health system, confidence monitoring.

MVP Definition of Done (gates, §16): G1 (0 xruns, 32ch, multi-hour), G2 (kill-9 → valid files), G3 (LTC TimeReference sample-exact), G4 (device-pull during record → capture survives), G5 (no RSS/thermal drift); all output validated by BWF MetaEdit/ffprobe/iXML tools.

3. Milestone Roadmap

Milestone 1 — Complete, independently testable recorder core

- **Objective:** glitch-free multitrack capture→disk with safe operation, **no TC/metadata yet**, fully testable headless.
- **Included:** device select/map/missing-detect/hot-plug survival; RT firewall + lock-free FIFO + writer pool; **our** mono+poly WAV/RF64/BWF writer; ≥32ch; GPU meters; disk remaining+estimate; local-volume guard; Record-Safe + Lock; SessionModel + save/load (versioned); crash-valid files + basic recovery; Telemetry (FIFO HWM, callback p999).
- **Excluded:** LTC, metadata, naming templates, markers, rollover, presets, dashboard, playback, exports.
- **Risks:** raw-HAL device handling; our writer correctness (RF64/crash-valid); 32→128ch jitter; recovery correctness. (*R1–R7*)
- **Validation:** 1h→multi-hour 32ch soak (0 xruns); kill-9 recovery; device-pull survival; golden-file format check; RT-guard + TSan clean.
- **Complexity:** **XL** (the hardest, highest-risk milestone — the whole reliability spine).

Milestone 2 — Timecode + Metadata

- **Objective:** make recordings broadcast-usable (correct TC + metadata + names + markers).
- **Included:** TimeAuthority hardening; **LTC decode** (7 rates) + display + sample-accurate TimeReference + ToD fallback; basic metadata model + bext/iXML embedding; token **naming engine** + preview; **markers/notes** + CSV; keyboard shortcuts for record/stop/marker.
- **Excluded:** LTC drift/flywheel/quality; visual naming builder; templates/presets; rollover; dashboard.
- **Risks:** audio↔TC correlation accuracy; iXML/bext correctness; naming collision safety. (*R5, §8/§9/§10*)
- **Validation:** LTC corpus (all rates, midnight, dropout) → TimeReference exact; BWF MetaEdit/iXML validation; naming property tests; marker CSV round-trip.
- **Complexity:** **L**.

MVP ships at end of M2.

Milestone 3 — Workflow features (Phase 2)

- **Objective:** fast multi-show setup + long-form continuity.
- **Included:** pre-record buffer; auto rollover (duration/size/TC); session/track **templates + presets**; **take/roll/segment** auto-numbering; **daily folders**; visual drag-drop naming builder; LTC drift/flywheel/quality; basic health + clip classification; marker PDF export.
- **Excluded:** mirror, dashboard, playback, exports, control APIs.
- **Risks:** gapless rollover sample-accuracy; counter/naming interplay; continuous-mode data model. (*§5/§7/§10*)
- **Validation:** rollover golden-files (no drop/dup at boundary); preset apply/round-trip; multi-day continuous-mode soak.
- **Complexity:** **L**.

Milestone 4 — Production-ready features (Phase 3a)

- **Objective:** field/broadcast hardening + operability.
- **Included:** full health monitoring (predictive FIFO/latency warnings, hysteresis, ack/snooze, session health summary); **mirrored recording**; full **recovery journal**; event log; **control surfaces** (OSC/HTTP API + Stream Deck via API); appliance hardening (auto-launch/recover, sleep/update inhibit).

- **Excluded:** dashboard, playback, exports (M5).
- **Risks:** mirror isolation; network trust boundary; multi-day reliability evidence. (*R2/R12/§13*)
- **Validation:** fault-injection matrix; 72h–7day soak with certification report; mirror-failure isolation test; control-API security/fuzz.
- **Complexity:** L–XL.

Milestone 5 — Advanced features (Phase 3b)

- **Objective:** convenience + remote awareness without risking capture.
- **Included:** out-of-process **remote dashboard** (read-only) + multi-recorder schema; isolated **playback/Review**; **exports** MP3/AAC/FLAC (out-of-process); confidence monitoring (if approved).
- **Excluded:** A/B chase, MIDI, editing (Cut).
- **Risks:** process isolation correctness; device contention (playback); export never mutates originals. (*§13/§14*)
- **Validation:** dashboard-crash → capture unaffected; playback device-handoff glitch-free; export bit-identical originals preserved.
- **Complexity:** M–L.

4. Module Dependency Map

(From hardened §3. **MVP?** = needed for M1+M2. **Postpone?** = can ship after MVP.)

Module	Purpose	Depends on	MVP?	Postpone?
AppCoordinator	composition root; DI; start/stop ordering; boot→recovery→acquire→restore	(all, via interfaces)	Yes (M1)	No
PlatformAbstraction (interfaces + mocks)	AudioDevice/FileSystem/DiskSpace/SystemClock/Storage/Net seams	—	Yes (M1)	No
AudioDeviceManager	device discovery/select/map/missing/hot-plug/clock; raw-HAL where needed	Platform	Yes (M1)	No
CaptureEngine	RT callback: delay→gain→bypass→meter/clip tap→FIFO; consume command FIFO	DeviceMgr, FIFO, TimeAuthority, Telemetry	Yes (M1)	No
LockFreeFIFO / CommandFIFO / Snapshot / EventBus	RT↔non-RT plumbing	—	Yes (M1)	No
Transport/TimeAuthority	sample master + sample↔TC↔wall map	—	Yes (M1 basic / M2 full)	No
DiskWriterPool	drain→format→WAV/RF64/BWF/iXML→crash-valid headers→preallocate	FIFO, our FormatWriter, VolumePolicy	Yes (M1)	No
FormatWriter (our WAV/RF64/BWF/iXML)	byte-level file format + crash-valid + metadata embed	TimeAuthority, Metadata	Yes (M1 WAV/RF64/BWF; M2 iXML/meta)	No
Destination/VolumePolicy	local-only enforce; cloud/network refuse; disk planning/remaining-time	Platform/DiskSpace	Yes (M1)	No
RecordingEngine	arm/start/stop; segment lifecycle; decides rollover; sample↔TC stamping	CaptureEngine, WriterPool, TimeAuthority	Yes (M1)	No
Telemetry	lock-free counters/histograms (FIFO HWM, callback p999, write latency)	—	Yes (M1)	No
RecoveryJournal	durable WAL; segment status; repair-on-launch	Storage(SQLite)	Yes (M1 basic / M4 full)	Partial
SessionModel	in-memory single source of truth + change-notify	—	Yes (M1)	No
SessionStore	persistence (JSON, versioned + migration)	SessionModel	Yes (M1)	No
MeteringService	aggregate RT meter atoms → UI rate	CaptureEngine taps	Yes (M1)	No
CommandService	the choke point; Record-Safe/Lock gating	SessionModel, RecordingEngine	Yes (M1)	No
GUI/ViewModels	bind snapshots/events; send commands; GPU meters; Dark/Light	CommandService, Snapshots	Yes (M1)	No
LTCEngine	decode (libltc), TC tracking, events	aux/selected channel tap, TimeAuthority	Yes (M2 core)	Partial (drift→M3)
MetadataService	field model, templates, live-edit, bext/iXML map	SessionModel	Yes (M2 basic)	Partial
NamingEngine	token resolution, sanitize, dedupe, preview	Metadata, Counters, TimeAuthority	Yes (M2)	Partial (builder→M3)
CounterService	roll/take/segment + rules	SessionModel	M2 minimal / M3 full	Partial
MarkerService	TC-stamped markers/notes; CSV/PDF	TimeAuthority, RecordingEngine	Yes (M2; PDF→M3)	Partial
LocalControl	keyboard (+ local Stream Deck later) → CommandService	CommandService	M2 (keyboard)	Partial
PreRecordBuffer	rolling pre-roll ring	CaptureEngine	No	Yes (M3)
PresetService/TemplateService	session/track presets+templates	SessionStore	No	Yes (M3)
HealthMonitor	sample/level/hysteresis/clip-classify	Telemetry, EventBus	basic clip only	Yes (M3/M4)
EventLog	append-only operational log	EventBus, Storage	No	Yes (M3/M4)
MirrorWriter	best-effort secondary write	WriterPool (own FIFO)	No	Yes (M4)
NetworkControlGateway	OSC/HTTP ingress (auth/rate-limit) → CommandService	CommandService	No	Yes (M4)
RemoteDashboard	out-of-process read-only status	shared snapshot IPC	No	Yes (M5)

Module	Purpose	Depends on	MVP?	Postpone?
PlaybackEngine	isolated spot-check; own output device	SessionModel, ledger	No	Yes (M5)
ExportService	out-of-process transcode from originals	ffmpeg child	No	Yes (M5)

Critical-path (must exist before anything records): PlatformAbstraction → AudioDeviceManager → CaptureEngine + FIFO + TimeAuthority(basic) → FormatWriter + WriterPool + VolumePolicy → RecordingEngine → CommandService → SessionModel/Store → GUI.
No MVP module depends on a postponed module.

5. Recommended Repository Structure

Key principle: the engine library has zero JUCE-UI dependency so it builds + tests headless in CI; the app target adds JUCE UI on top.

```
recorder/
├─ CMakeLists.txt           # top-level; options: BUILD_APP, BUILD_TESTS, BUILD_BENCH
├─ cmake/                  # toolchain, deps (FetchContent/pinned), Audio Workgroups helpers
├─ third_party/            # pinned: JUCE, libltc, sqlite (vendored or FetchContent)
├─ engine/                 # ← pure C++ core, NO JUCE-UI. The reliability spine.
│  └─ rt/                  # RT firewall, lock-free FIFO/command/snapshot, RT-guard
│  └─ audio/               # CaptureEngine, device abstraction, meter/clip taps
│  └─ time/                # TimeAuthority, LTC engine
│  └─ io/                  # FormatWriter (WAV/RF64/BWF/iXML), WriterPool, VolumePolicy
│  └─ record/              # RecordingEngine, segment lifecycle, rollover
│  └─ session/             # SessionModel, metadata, naming, counters, markers
│  └─ recovery/            # journal, repair-on-launch
│  └─ telemetry/           # counters/histograms
│  └─ platform/            # interfaces (AudioDevice/FileSystem/DiskSpace/Clock/Storage/Net)
│     └─ macos/            # raw-HAL CoreAudio, APFS, Audio Workgroups impls
├─ app/                   # ← JUCE app: GUI/ViewModels, CommandService wiring, LookAndFeel
│  └─ ui/                  # views, GPU meter view, Dark/Light themes
│  └─ control/             # keyboard; (later) OSC/HTTP gateway, Stream Deck
├─ services/              # out-of-process helpers (later): dashboard/, exporter/
├─ tests/
│  └─ unit/                # pure logic (naming, counters, time math, metadata)
│  └─ lockfree/            # TSan stress for FIFO/command/snapshot
│  └─ integration/         # mock-device → FIFO → writer → file
│  └─ format_golden/       # WAV/RF64/BWF/iXML vs external tools
│  └─ ltc_corpus/          # synthetic LTC fixtures + decode assertions
│  └─ fault_injection/     # device pull, SR change, disk full, kill-9
│  └─ soak/                # 1h/24h/72h/7d harness + invariant checks
│  └─ fixtures/            # seedable mock device, fake disk, golden files
├─ tools/                  # md2pdf, golden regenerators, soak report generator
├─ docs/
│  └─ architecture/       # Part I (§§1–21) split per section + reviews/
│  └─ execution/           # this Part II
│  └─ runbooks/            # appliance setup, recovery drills (M4)
└─ .github/workflows/     # CI: headless engine build+tests; perf CI; (mac runner for goldens)
```

Build strategy: CMake + pinned deps via FetchContent; three targets — recorder_engine (lib, headless, CI-tested), recorder_app (JUCE), recorder_tests . Debug builds enable the **RT-guard** + sanitizers (ASan/TSan/UBSan); Release tunes for Apple Silicon + Audio Workgroups. Perf/soak as separate CTest labels.

Session/project file strategy (from §5): two stores — **mutable config** = human-readable **JSON**, **versioned with forward-migration** (.recession bundle: session.json + templates + presets); **immutable recording ledger** = **SQLite WAL** (segments/markers/events) for crash-safe durability + recovery. A session is a **bundle/folder** (config JSON + ledger.db + daily folders), not a single opaque file — survivable and inspectable.

6. Testing Plan

Layer	What	How / tools
Unit	NamingEngine (sanitize/dedupe/preview), CounterService, TimeAuthority rational math, Metadata mapping, VolumePolicy	property + example tests, headless, fast
Lock-free correctness	FIFO/command/snapshot under contention	ThreadSanitizer + stress loops; ABA/ordering assertions
RT-safety	no alloc/lock/syscall in callback; jitter	RT-guard trap (debug) + callback p999 / FIFO HWM harness @ 32–128ch
Integration	mock-device → delay/gain → FIFO → writer → file	seedable fake device + fake disk; assert sample-exact output
Long-duration recording	1h / 24h / 72h / 7-day continuous	soak harness; invariants: 0 xruns, RSS flat, thermal bounded, every file valid; automated pass/fail + report artifact
LTC dropout	dropout→flywheel→recovery, midnight, reverse, varispeed, DF/NDF, level	synthetic LTC corpus; assert TimeReference exactness + correct state events
Device disconnect/reconnect	pull/re-add during arm, record, rollover	mock + hardware (Dante/RME) fault injection; capture survives, gap marked, re-acquires
Disk failure simulation	disk full (start/mid/near-4GB), slow disk, I/O error, removed volume	fake failing FileSystem + real external SSD; mirror continues, primary finalizes valid, CRITICAL raised
RF64 validation	>4GB transition, ds64, header correctness, WAV↔RF64 decision	golden-file compare vs BWF MetaEdit + ffprobe + sox ; large-file generation test
Recovery	kill-9 / power-loss across fault×state matrix	repair-on-launch → 100% files openable (G2)
Format goldens	WAV/RF64/BWF/iXML correctness	external tools in CI; documented regen+review
Perf regression	callback jitter, write latency, CPU/channel over time	perf CI; regressions block merge
UI	ViewModel/snapshot binding; 60Hz meter render no-alloc	headless VM tests + manual checklist

SLO gate matrix (§16) wires G1–G5 to the specific suites above; a release requires all gates green + a soak certification report.

7. First Coding Milestone (M0 — the foundation slice)

- Smallest useful thing that compiles, runs, is independently testable, and locks in the architecture — before real CoreAudio or any UI.**
- Scope:** top-level CMake + three targets; `engine/rt` (RT-guard + lock-free FIFO + triple-buffer snapshot + command FIFO); `engine/platform` interfaces + **seedable mock audio device** + **fake disk**; minimal `FormatWriter` (mono+poly **WAV**, then RF64 header) + single-thread writer; a headless `RecordingEngine` path: **mock device** → **FIFO** → **writer** → **WAV file**; Telemetry counters; a CTest suite (unit + integration + lockfree-TSan + one format-golden).
 - Explicitly NOT in M0:** real CoreAudio, GUI, LTC, metadata, BWF/iXML, recovery, multi-thread writer pool tuning.
 - Compiles:** `cmake --build` produces `recorder_engine` + `recorder_tests`.
 - Runs:** a tiny headless harness records 30s from the mock device to a valid WAV.
 - Testable independently:** `ctest` passes unit + integration + TSan + golden, with the **RT-guard clean**.
 - Establishes the architecture:** RT/non-RT firewall, lock-free plumbing, platform mock seams, our-writer ownership, headless-engine-for-CI — every later module plugs into these.
 - Exit criterion:** mock-device→WAV integration test green under ASan/TSan, RT-guard reports zero violations, golden WAV validates in `ffprobe`.

M0 is a strict subset of Milestone 1. After M0, the next slice swaps the mock device for **real CoreAudio (raw-HAL)** and adds the **writer pool** + **crash-valid headers** + **VolumePolicy** — completing M1 incrementally.

8. Recommended Claude Code Session Sequence

Principles to minimize tokens & avoid architectural drift:

- One session = one module/concern**, scoped to a single milestone slice; don't reopen settled design.
- Start each session** by reading only the relevant planner section(s) + the interface contracts it touches (not the whole planner) — point Claude at `docs/architecture/<section>.md` and the specific headers.
- End each session** with: tests green, RT-guard/sanitizers clean (where applicable), a focused **commit**, and a 3-line "what changed / what's next" note appended to `docs/execution/progress.md`.
- Contracts before implementations:** lock interfaces/headers first; implement against them; this prevents drift across sessions.
- Keep context small:** prefer many short sessions over one long one; let earlier commits + the progress note carry state instead of re-reading everything.

#	Session	Goal (deliverable)	Reads	Gate
S0	Repo + CI scaffold	move out of OneDrive; CMake 3 targets; CI headless; pinned deps; RT-guard skeleton	§15, §17, repo-struct	builds + CI green
S1	RT plumbing	lock-free FIFO + command FIFO + triple-buffer snapshot + RT-guard	§2, §6	TSan stress green
S2	Platform seams + mocks	interfaces + seedable mock device + fake disk	§2, §3, §6	mock unit tests
S3	FormatWriter (WAV/RF64)	our byte-level WAV→RF64 writer + golden test	§7, §9, §15	ffprobe golden
S4	M0 integration slice	mock→FIFO→writer→WAV headless + Telemetry	§6, §7, §21	M0 exit criteria
S5	Real device (raw-HAL)	CoreAudio acquire/map/missing/ hot-plug	§6, §15	hardware pull test
S6	WriterPool + crash-valid + VolumePolicy	pool partition, header patching, local-only, recovery basics	§5, §7	kill-9 recovery
S7	RecordingEngine + CommandService + Lock/Record-Safe	arm/start/stop, gating choke point	§2, §3, §11	gating tests
S8	SessionModel + Store (versioned)	save/load bundle, migration	§5	round-trip + migrate
S9	GUI shell + GPU meters + disk-time	track list, meters, Dark/Light	§11	32–128ch @60Hz no-alloc
S10	M1 soak + harden	wire SLO gates; multi-hour 32ch soak	§16, §18	M1 DoD (G1/G2/G4/G5)
S11	TimeAuthority + LTC core	decode 7 rates + TimeReference + ToD fallback	§8	LTC corpus exact (G3)
S12	Metadata + bext/iXML	field model + embedding	§9	BWF MetaEdit/iXML valid
S13	NamingEngine + Markers + keyboard	tokens/preview + TC markers + CSV	§10, markers	property + CSV round-trip
S14	M2 soak + MVP cert	full MVP DoD + certification report	§16, §18, §19	MVP gates green
...	M3→M5 sessions	one module per session per the dependency map	resp. sections	per-milestone validation

Drift guards across sessions: (1) interfaces are the contract — change them only in a dedicated "interface change" session that updates dependents; (2) every session re-states the relevant SLO/gate it must not regress; (3) `progress.md` + commits are the source of truth between sessions; (4) never let a feature session weaken a Tier-0 invariant — if tempted, stop and open a design note instead.

PART II Status

Execution plan complete: feature triage, realistic MVP (= M1+M2), 5-milestone roadmap, module dependency map, repo structure, testing plan, first coding milestone (M0), and a 15+ session Claude Code build sequence. Honors the user's MVP capability list while keeping reliability first. Still **no code**. Confidence-monitoring scope **resolved** → **M5** (MVP metering-only).

PART III — S0 IMPLEMENTATION BRIEF (M0 Foundation Slice)

Scope: S0 == the M0 foundation slice. The smallest independently testable foundation that locks in the architecture **before any CoreAudio or UI. No code in this brief.** Implementation begins only after scope approval, using the prompt in §10 below.

Stack baseline: C++20, CMake ≥ 3.24, clang (Apple Silicon), GoogleTest + CTest, FetchContent for deps. Sanitizers via build options. **Zero JUCE / zero UI** in S0.

1. Exact S0 Goals

- G-S0.1** A CMake project producing a pure C++20 static lib `recorder_engine` (no JUCE, no UI, no platform-audio API) plus a `recorder_tests` target wired into CTest.
- G-S0.2 Platform abstraction interfaces** (audio device, clock, file sink) with **deterministic mock implementations** — no real hardware/OS audio.

- **G-S0.3** A **lock-free SPSC ring buffer** (audio FIFO) with correct acquire/release memory ordering, fixed-capacity, allocation-free in steady state.
- **G-S0.4** An **RT-safety guard**: debug instrumentation that flags heap allocation occurring while a thread is in "real-time context"; compiled out in release.
- **G-S0.5** A **minimal audio file writer**: WAV (canonical) with **automatic RF64 promotion** at the 4 GB threshold; supports **mono (one file/track)** and **polyphonic (interleaved)**; sample format = **32-bit float** for S0.
- **G-S0.6** A **headless mock audio device** that synthesizes a deterministic, verifiable signal and drives a render callback, with a **synchronous pump mode** (test-driven block count) and an optional **threaded mode** (for TSan stress).
- **G-S0.7** A **mock-device** → **FIFO** → **writer** → **.wav integration test** asserting sample-accurate, gap-free output and a valid parsed header.
- **G-S0.8** **ASan/UBSan and TSan** build configurations; all tests green under sanitizers in CI; headless and fast (< ~30 s).
- **G-S0.9** A **CI-friendly structure** (headless build+test job(s); fprobe-based golden validation where available).

2. Non-Goals (explicitly out of S0)

- No real **CoreAudio** / raw-HAL (that's S5). No **JUCE** / **GUI**.
- No **LTC**, **metadata**, **BWF/iXML**, **naming**, **markers**, **sessions**, **recovery**, **telemetry dashboards**.
- No **per-track delay/gain/bypass** processing — S0 capture is **pass-through** (processing path is a later session).
- No **multi-thread writer pool tuning** — a **single writer/drain path** is sufficient for S0.
- No **device hot-plug**, **sample-rate renegotiation**, **mirror**, **networking**, **exports**.
- No **int16/int24 PCM** writer yet (32-bit float only in S0; int24 broadcast format is the next writer session).
- No **128-channel performance tuning** — correctness at a modest count (e.g., 1–32 ch) is the bar; the path must be *architected* for scale, not optimized.

3. Folder Structure (created/touched in S0)

```
recorder/
├── CMakeLists.txt
├── cmake/
│   └── # sanitizers.cmake, warnings.cmake, deps.cmake
├── engine/
│   ├── CMakeLists.txt
│   │   └── # builds recorder_engine (static lib)
│   ├── rt/
│   │   └── # SpscRingBuffer, RtGuard, rt context flag
│   ├── platform/
│   │   ├── # interfaces: IAudioDevice, IClock, IFileSink
│   │   ├── └─ mock/
│   │   │   └── # MockAudioDevice, MockClock, MockFileSink/InMemory
│   │   ├── io/
│   │   │   └── # WavWriter (+ RF64 promotion), audio format types
│   │   └── capture/
│   │       └── # CapturePath (device→FIFO→writer wiring for S0)
├── tests/
│   ├── CMakeLists.txt
│   │   └── # recorder_tests (GoogleTest + CTest)
│   ├── unit/
│   │   └── # ring buffer, wav header, rf64 threshold
│   ├── lockfree/
│   │   └── # SPSC TSan stress
│   ├── rtguard/
│   │   └── # guard self-test (planted allocation)
│   ├── integration/
│   │   └── # mock→FIFO→writer→wav
│   └── fixtures/
│       └── # signal generators, wav/ffprobe helpers
├── tools/
│   └── # (existing) md2html etc.
├── docs/
│   └── # (existing planner) architecture/ + execution/
└── .github/workflows/ci.yml
    └── # headless build + ASan/UBSan + TSan jobs
```

4. Files to Create (inventory — purpose only, no code)

Build: CMakeLists.txt (top), engine/CMakeLists.txt, tests/CMakeLists.txt, cmake/{sanitizers,warnings,deps}.cmake, .github/workflows/ci.yml. **RT:** rt/SpSCRingBuffer.hpp (fixed-capacity lock-free SPSC), rt/RtContext.hpp (thread-local in-RT flag + scope guard), rt/RtGuard.hpp/.cpp (operator new/delete override → violation counter when in-RT). **Platform interfaces:** platform/IAudioDevice.hpp, platform/IClock.hpp, platform/IFileSink.hpp, platform/AudioFormat.hpp (sample rate, channels, sample type, block size). **Mocks:** platform/mock/MockAudioDevice.hpp/.cpp (deterministic signal, pump + threaded modes), platform/mock/MockClock.hpp, platform/mock/InMemoryFileSink.hpp/.cpp (+ real-file sink for golden). **IO:** io/WavWriter.hpp/.cpp (WAV + RF64 promotion, mono/poly), io/AudioFileTypes.hpp. **Capture wiring:** capture/CapturePath.hpp/.cpp (device callback → ring buffer → drain → writer). **Tests:** unit/SpSCRingBufferTest, unit/WavWriterTest, unit/Rf64PromotionTest, lockfree/SpSCStressTest, rtguard/RtGuardTest, integration/MockToWavTest, fixtures/SignalGen + fixtures/WavProbe.

5. Interfaces to Define (contracts, prose only)

- **AudioFormat** — value type: sampleRate, numChannels, sampleType (float32 for S0), maxBlockSize. Immutable once a device starts.

- **IAudioDevice** — `setCallback(renderFn)`, `start()`, `stop()`, `getFormat()`. Render signature delivers **non-interleaved** channel pointers + a frame count $\leq \text{maxBlockSize}$. **Contract:** the render fn runs in **RT context**; it must not allocate/lock/block. Mock adds `pump(blocks)` (sync) and `runThread(blocks)` (async).
- **IClock** — `nowSamples()` / `nowMonotonic()`; mock is fully controllable/deterministic. (Seeds the future TimeAuthority.)
- **IFileSink** — `open(path)`, `write(bytes)`, `flush()`, `close()`, `bytesWritten()`. Implementations: real-file + in-memory (for header-byte assertions). Writers depend on `IFileSink`, not on the OS.
- **SpScRingBuffer<T>** — fixed capacity at construction (no runtime alloc); `push(span)` / `pop(span)` returning counts; `availableToRead/Write()`; single-producer/single-consumer; explicit acquire/release ordering. **Contract:** `push` is RT-safe (producer = RT thread); `pop` runs on the drain/writer thread.
- **RtContext** / **RtGuard** — `RtContextScope` sets the thread-local in-RT flag for the duration of the callback; `RtGuard` counts/asserts allocations while the flag is set. Release builds compile both to no-ops.
- **IAudioFileWriter (WavWriter)** — `begin(format, path, mode=mono|poly)`, `writeBlock(channelData, frames)`, `finalize()`. **Contract:** chooses WAV vs RF64 header up front when size is projected $\geq$ threshold, and **promotes/patches to RF64 + ds64** when crossing 4 GB; `finalize` patches sizes so the file is valid.
- **CapturePath** — composes `IAudioDevice` + `SpScRingBuffer` + writer: device render pushes to the ring (RT side); a drain step pops and writes (non-RT side). S0 drain may be pumped by the test for determinism.

6. Tests to Write

- **Unit — SpScRingBuffer:** empty/full, wrap-around, partial push/pop, capacity invariants, no allocation after construction.
- **Unit — WavWriter:** byte-exact canonical WAV header (RIFF/fmt/data) for mono & poly float32; round-trip written samples == input; `flush` / `finalize` size fields correct.
- **Unit — RF64 promotion:** with a **lowered threshold** (injected, not 4 GB of real data), confirm header becomes RF64 + `ds64` and remains valid; boundary just-below stays WAV.
- **Lock-free — SPSC stress (TSan):** producer thread + consumer thread for N seconds/iterations; assert no data race, no lost/duplicated samples, FIFO never corrupts.
- **RT-guard self-test:** a callback that **deliberately allocates** → guard reports a violation; the real capture callback → **zero** violations.
- **Integration — MockToWav:** mock device emits a known signal (e.g., per-channel ramp/sine with channel-ID watermark) → `CapturePath` → WAV; reopen and assert **sample-accurate, gap-free, correct channel order** for both mono and poly; header parses; (where `ffprobe` present) **golden validation**.
- **Determinism:** same seed ⇒ byte-identical output file.

7. Acceptance Criteria

- **A1** `cmake --build` produces `recorder_engine` + `recorder_tests` with **zero warnings** (warnings-as-errors) and **no JUCE/UI/OS-audio dependency** in the engine lib (verifiable by link inputs).
- **A2** `ctest` — **all green** in normal, **ASan+UBSan**, and **TSan** configurations.
- **A3** RT-guard: **0 violations** in the real capture path; **catches** the planted allocation in its self-test.
- **A4** Integration test yields a **bit-exact, gap-free** WAV (mono + poly) matching the generated signal; header parses; `ffprobe`-valid where available.
- **A5** RF64 promotion path produces a **valid RF64+ds64** header at the injected threshold; sub-threshold stays canonical WAV.
- **A6** Runs **headless** in CI, no hardware, < **~30 s** total; deterministic (seeded ⇒ identical bytes).
- **A7** Public engine headers expose **only the S0 interfaces** above (no leakage of mock/impl details).

8. Risks

- **R-S0.1 SPSC memory-ordering bugs** → subtle data races. *Mitigate:* TSan stress + careful acquire/release; keep the API tiny.
- **R-S0.2 RT-guard portability** (overriding global `operator new` / interposing `malloc` on macOS) → may miss some alloc routes or affect the test process broadly. *Mitigate:* thread-local in-RT flag scoping; guard only asserts when flag set; document limits; keep it debug-only.
- **R-S0.3 RF64 header correctness** (ds64, size patching) → invalid files. *Mitigate:* in-memory sink byte assertions + `ffprobe` golden + threshold-injection test (avoid writing real 4 GB).
- **R-S0.4 Mock-device threading vs determinism** → flaky tests. *Mitigate:* default **synchronous pump** for the integration test; threaded mode only for the TSan stress.
- **R-S0.5 ASan+TSan can't combine** → must be **separate** build configs/CI jobs. *Mitigate:* two CMake presets/CI matrix entries.
- **R-S0.6 ffprobe absent in CI** → golden step skipped. *Mitigate:* make golden validation conditional; keep byte-level header asserts as the always-on guarantee.
- **R-S0.7 Float32-only now** could mask int24 issues later. *Accepted:* int24 writer is an explicit next session; S0 proves plumbing.

9. Estimated Order of Implementation

1. **CMake skeleton** (top + engine + tests targets, FetchContent GoogleTest, warnings-as-errors). (S)
2. **Sanitizer presets** (ASan+UBSan config, TSan config) + a trivial passing test to prove CI. (S)
3. **SpSCRingBuffer** + unit tests. (M)
4. **RtContext** + **RtGuard** + self-test. (S-M)
5. **Platform interfaces** (`AudioFormat` , `IAudioDevice` , `IClock` , `IFileSink`). (S)
6. **Mocks** (MockClock, InMemory/real FileSink, MockAudioDevice with pump + threaded). (M)
7. **WavWriter** (canonical WAV mono/poly) + unit/header tests. (M)
8. **RF64 promotion** + threshold-injection test. (M)
9. **CapturePath** wiring + **MockToWav integration test** (mono + poly, determinism). (M)
10. **SPSC TSan stress** + final **CI matrix** (normal/ASan/TSan, conditional ffpromote golden). (M)

Rough total: **M (single focused implementation session)**; steps 3, 7, 8, 9 carry the real risk.

10. Next Claude Code Prompt (paste after approving this scope)

Implement S0 / M0 (the foundation slice) exactly per PART III of docs/execution (the S0 Implementation Brief). No CoreAudio, no JUCE, no UI.

Read first: PART III §§1–9, plus Part I §2 (firewall/planes), §6 (audio pipeline callback contract), §7 (WAV/RF64/crash-valid writer), and the repo-structure section.

Build, in the order in §9:

1. CMake project: recorder_engine (pure C++20 static lib, zero JUCE/UI/OS-audio deps) + recorder_tests (GoogleTest via FetchContent, CTest), warnings-as-errors, plus cmake/ presets for ASan+UBSan and TSan (separate configs).
2. engine/rt: SpSCRingBuffer<T> (fixed capacity, lock-free SPSC, acquire/release), RtContext (thread-local in-RT flag + scope) and RtGuard (operator new/delete override counting allocations while in-RT; no-op in release).
3. engine/platform: interfaces AudioFormat, IAudioDevice, IClock, IFileSink, and mocks (MockAudioDevice with pump() + runThread(), MockClock, InMemoryFileSink + real-file sink).
4. engine/io: WavWriter (canonical WAV, float32, mono + interleaved poly) with automatic RF64+ds64 promotion at an injectable size threshold; finalize patches sizes for a valid file.
5. engine/capture: CapturePath wiring device-render → SpSCRingBuffer → drain → WavWriter (drain pump-able by tests).

Write all tests in §6 and meet every acceptance criterion in §7:

- ctest green in normal, ASan+UBSan, and TSan configs
- RtGuard: 0 violations in the capture path; catches a planted allocation
- MockToWav integration: bit-exact, gap-free WAV (mono + poly), header parses, ffpromote-valid where available; seeded ⇒ identical bytes
- RF64 promotion valid at the injected threshold

Honor the firewall: nothing in the render callback allocates/locks/blocks. Keep public engine headers limited to the S0 interfaces.

When done: ensure all sanitizer configs pass, then STOP – do not add features beyond S0. Append a 3-line entry to docs/execution/progress.md (what changed / tests status / next = S5 real CoreAudio). Do NOT commit unless I ask.

PART III Status

S0/M0 brief complete: goals, non-goals, folders, file inventory, interface contracts, tests, acceptance criteria, risks, implementation order, and the ready-to-run next prompt. Awaiting scope approval before any code.